INTERACTIVE SINHALA TEXT READER FOR THE VISUALLY IMPAIRED USING OCR, NLP AND AI

INDIVIDUAL THESIS

Sathsara S.P.Y.B

(IT22004468)

Adaptive Conversational Personalization and Voice Interaction Module

Group ID: R26-DS-002

Report submitted in partial fulfillment of the requirements for the
Bachelor of Science (Hons) in Information Technology, specialising in Data Science

Department of Computer Science

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION

I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Sathsara S.P.Y.B (IT22004468)

Signature: Date:

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Signature of the supervisor: Date:
Dr. Lakmini Abeywardhana (Supervisor)

Signature of the co-supervisor: Date:
Ms. Mathishi Adya Dissanayake (Co-supervisor)

Abstract

Visually impaired and low-vision Sinhala speakers depend on assistive readers that read text aloud but cannot hold a conversation or adapt to the listener. Every user gets the same output, and Sinhala lacks the labelled data that most personalisation and intent detection methods need. This thesis presents the voice interaction and personalisation component of a group-built Sinhala assistive reader. The component converts a spoken Sinhala request into a personalised prompt for the group's answer-generation component and speaks the answer through the system's text-to-speech. Following a design science approach, each stage was selected by comparing alternatives on my own recordings. Google speech recognition gave 0% word error rate on the test commands, while Whisper produced no usable Sinhala. Translating Sinhala to English with NLLB-200 and extracting intent with a local Llama 3.2 1B model achieved 77.78% intent accuracy on 45 Sinhala test commands without any Sinhala intent dataset, compared with 2.22% for direct Sinhala input to a small model (Qwen2.5 1.5B) and 62.22% for a trained TF-IDF classifier. The main contribution is an evidence-only, per-user online learning method for answer style (Simple, Detailed or StepByStep). Each user has a River model that combines the words of the request with a recency-weighted summary of their confirmed history. Explicit requests always decide the current turn, style flags from the language model are accepted only when the user's own words support them, and the model never learns from its own predictions. A first version that also learned from its own predictions collapsed to Simple in about 85% of logged interactions. In a scripted test of the final design, users who taught different preferences received Simple, Detailed and StepByStep answers to the same neutral question, each decided by their own model, starting from no data. Formal user evaluation and adaptive speech output remain future work.

Keywords: Sinhala voice interaction, personalisation, online learning, intent detection, assistive technology

ACKNOWLEDGEMENT

I am sincerely grateful to my supervisor, Dr. Lakmini Abeywardhana, for her guidance throughout this research and I also thank my co-supervisor, Ms. Mathishi Adya Dissanayake, for her support and feedback.

I thank the members of my research group, who built the reading, error correction and answer generation components that my work connects to, for their cooperation during integration. I am grateful to the visually impaired person who took the time to listen to my prototype and share how they would like to talk to such a system, and to the members of the evaluation panels, whose comments shaped the direction of this component.

Finally, I thank my family and friends for their patience and encouragement throughout the year.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 2

1.4 Research gap 3

1.5 Aim and objectives 4

1.6 Research questions or hypotheses 5

1.7 Scope limitations and assumptions 5

1.8 Individual contribution 6

1.9 Thesis structure 7

2 LITERATURE REVIEW AND RELATED WORK 8

2.1 Review method and scope 8

2.2 Domain and theoretical foundations 8

2.3 Related research 10

2.4 Existing systems products and techniques 11

2.5 Gap synthesis and conceptual framework 12

3 RESEARCH METHODOLOGY 14

3.1 Research design 14

3.2 Data participants or study materials 14

3.3 Experimental or study procedure 15

3.3.1 Speech-to-text comparison 15

3.3.2 Intent detection comparison 16

3.3.3 Personalisation evaluation 17

3.3.4 Diagnosis of the first personalisation version 17

3.4 Measures and evaluation metrics 18
3.5 Data analysis 18
3.6 Ethics privacy security and responsible AI 19
3.7 Reproducibility and research artefacts 20
4 PRODUCT AND SOLUTION DESIGN 21
4.1 Requirements 21
4.2 Component architecture within the complete solution 22
4.3 Design alternatives and decisions 24
4.4 Detailed design 26
4.4.1 Intent and flag extraction 26
4.4.2 System commands 27
4.4.3 The style decision 27
4.4.4 Correction 28
4.4.5 The per-user model 29
4.4.6 Prompt modifiers and the output contract 31
4.4.7 Interaction log 32
4.5 Standards and professional practice 32
5 IMPLEMENTATION INTEGRATION AND TESTING 34
5.1 Development environment and technology stack 34
5.2 Component implementation 34
5.3 Integration with the other components 36
5.4 Verification and software testing 37
5.5 Deployment and operation 39
6 RESULTS AND VALIDATION 40
6.1 Research results 40
6.1.1 Speech to text (RQ1) 40

6.1.2 Intent detection (RQ2) 41
6.1.3 Personalisation (RQ3 and RQ4) 43
6.2 Product and technical results 47
6.3 User or stakeholder validation 48
6.4 Integration results 49
6.5 Summary of findings 49
7 DISCUSSION 50
7.1 Interpretation of findings 50
7.2 Comparison with literature baselines and existing solutions 52
7.3 Research contribution 53
7.4 Product contribution 53
7.5 Limitations and threats to validity 54
7.6 Implications 55
8 COMMERCIALISATION AND SUSTAINABILITY 56
8.1 Value proposition and target users 56
8.2 Market and competitive position 56
8.3 Business or sustainability model 57
8.4 Scalability intellectual property and compliance 57
8.5 Contribution to the Sustainable Development Goals 58
9 CONCLUSION AND FUTURE DIRECTIONS 59
9.1 Conclusion 59
9.2 Achievement of objectives 60
9.3 Individual research and product contributions 60
9.4 Future directions 61
REFERENCES 62
GLOSSARY 65

APPENDICES 67
Appendix A Research instruments and extended results 67
Appendix B Academic integrity and AI use disclosure 72

LIST OF TABLES

Table 2.1 Techniques considered for each stage of the component 12
Table 3.1 Data and materials used in the research 15
Table 3.2 Evaluation metrics 18
Table 4.1 Functional requirements 21
Table 4.2 Non-functional requirements 21
Table 4.3 Design alternatives and decisions 24
Table 4.4 The four-tier style decision 28
Table 4.5 Worked example of the history scorecard update (decay 0.85) 30
Table 4.6 Personalisation settings 31
Table 4.7 Prompt modifiers sent to Component 3 31
Table 4.8 Interaction log record 32
Table 5.1 Technology stack 34
Table 5.2 Verification test cases 38
Table 6.1 Speech-to-text comparison on my Sinhala recordings 40
Table 6.2 Intent detection approaches on the 45 Sinhala test commands 41
Table 6.3 Classifier alternative on the generated command set (530 sentences, 17 intents) 43
Table 6.4 Three-user personalisation test across runs 44
Table 6.5 Final three-user run on the local model (from the interaction log) 44
Table 6.6 Style evidence for the seven recorded test commands 46
Table 6.7 Defects found during testing and how they were fixed 47
Table 7.1 Comparison with related work 52
Table 9.1 Achievement of objectives 60
Table A.1 Intent test commands 67
Table A.2 Three-user teaching script 69
Table A.3 Approach 1 output for each test command (intent comparison run) 69
Table B.1 Use of generative AI and automated tools 72

LIST OF FIGURES

Figure 4.1 Component 4 within the group system 22
Figure 4.2 Per-turn flow of Component 4 23
Figure 4.3 Structure of the per-user style model and its learning gate 29
Figure 6.1 Accuracy and average latency of the intent approaches on the 45 test commands 42

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ASR	Automatic Speech Recognition
CER	Character Error Rate
CPU	Central Processing Unit
CSV	Comma-Separated Values
CTC	Connectionist Temporal Classification
DSR	Design Science Research
GGUF	GPT-Generated Unified Format (quantised model file format used by llama.cpp)
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
NLLB	No Language Left Behind (translation model)
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDPA	Personal Data Protection Act
RAG	Retrieval-Augmented Generation
RQ	Research Question
SDG	Sustainable Development Goal
STT	Speech-to-Text
TF-IDF	Term Frequency-Inverse Document Frequency
TTS	Text-to-Speech
WER	Word Error Rate

LIST OF APPENDICES

Appendix A Research instruments and extended results 67
Appendix B Academic integrity and AI use disclosure 72

1 INTRODUCTION

1.1 Overall project context

Printed and digital text is still the main way people get everyday information, such as newspapers, notices, letters and school material. For a person who cannot see well, most of that text is out of reach unless someone reads it aloud. The World Health Organization estimates that at least 2.2 billion people live with a near or distance vision impairment, and that around half of these cases have not been prevented or addressed [1]. In Sri Lanka, the majority of the population reads and speaks Sinhala, but many widely used assistive reading tools are designed around English. Sinhala is also a low-resource language for language technology, which means there is far less data and far fewer ready-made tools for it than for English [2], [3].

Our group project, "Interactive Sinhala Text Reader for the Visually Impaired Using OCR, NLP and AI", tries to close this gap with one complete system. A visually impaired or elderly low-vision user captures a page with a phone camera, and the system reads it, understands it and talks about it with the user in Sinhala. The work is split into four individual components:

- **Component 1: Topological Sinhala character reader.** Finds the article layout on the page and reads the headline text, including stylised Sinhala headline fonts.
- **Component 2: Context-aware post-OCR error correction.** Reads the body text and corrects OCR errors with a fine-tuned transformer.
- **Component 3: Document retrieval and information generation.** Stores the processed articles and answers user questions with retrieval-augmented generation (RAG).
- **Component 4: Adaptive conversational personalization and voice interaction (this thesis).** Lets the user talk to the system in Sinhala, understands what they want, learns how each user likes their answers, and passes a personalised request to Component 3.

The first three components turn a page into clean, searchable text. Component 4 is the part the user actually talks to.

1.2 Individual component context

Component 4 is the voice front end of the reader. A user speaks a request such as "මෙය සාරාංශ කරන්න" ("summarise this") or "තව විස්තර කරන්න" ("explain more"). The component has to:

1. turn the Sinhala speech into text,
2. work out what the user wants (the intent),
3. decide how this particular user likes to receive answers (the answer style),
4. build a structured, personalised request for the answer-generation component (Component 3), and
5. hand the answer to the speech output so the user hears it.

The research focus of my component sits in step 3. Speech-to-text, intent detection and text-to-speech are needed to make the module work, but there are already many established ways to do them. The open question for Sinhala users is personalisation: the same request should not get the same kind of answer for every person. One user wants a short and simple explanation, another wants every detail, and a third prefers to hear things one step at a time. My component learns this preference for each user from that user's own interactions, while the system is being used, without any training dataset collected in advance.

The final pipeline built in this research is:

Google Speech-to-Text (Sinhala) → NLLB-200 translation to English → Llama 3.2 1B intent and flag extraction (run locally) → system-command filter → per-user answer-style decision with River online learning → personalised JSON prompt to Component 3 → spoken answer through the system's text-to-speech.

1.3 Problem statement

Current assistive reading systems treat the visually impaired user as a listener, not as someone having a conversation. They convert text to speech and play it, and every user gets the same output in the same way. Commercial voice assistants are

conversational, but they are built for general tasks in widely used languages and do not adapt the way they explain things to an individual user [4]. Sinhala speech research has mostly focused on recognition accuracy and fixed command sets [5], [6], so the step after recognition, which is understanding a free-form request and responding in a way that suits the person, is largely missing.

This creates three practical problems for a Sinhala-speaking visually impaired user:

- They cannot ask for what they want in their own words. Command-based systems need the user to remember fixed phrases.
- The system does not remember them. Even if a user asks for simple explanations every single time, the next answer starts from zero again.
- There is very little labelled Sinhala data to train such a system. A personalisation model cannot depend on a large dataset of Sinhala users that does not exist.

The problem this component addresses is therefore: **how can a low-cost Sinhala voice interface understand free-form spoken requests and learn each user's preferred answer style from their own interactions, without an upfront training dataset and without overriding what the user explicitly asks for?**

1.4 Research gap

The literature (Chapter 2) shows four gaps that meet in this component:

- **Sinhala speech work stops at recognition.** Studies on Sinhala ASR report word error rates and command recognition, but do not continue into conversation or adaptation [5], [6].
- **Intent detection for Sinhala depends on labelled data.** Existing Sinhala intent work trains classifiers on fixed intent sets [6], [7]. Large public intent datasets exist only for languages like English [8].
- **Personalisation research rarely targets answer style.** Dialogue personalisation work mostly adapts a persona or a recommendation list [9], [10], not how detailed or how structured an explanation should be for a particular listener.
- **Online learners that learn from their own guesses drift.** Systems that retrain on their own predictions reinforce their own mistakes [11], [12]. For a

personalisation model that learns live, the question of which turns should count as training evidence is central, but it is rarely addressed in assistive voice systems.

To my knowledge, there is no Sinhala assistive voice interface that learns a per-user answer style online, from the user's own explicit signals, with no upfront dataset, and passes it to a generation model as a structured prompt. This is the gap my component targets.

1.5 Aim and objectives

Aim. To design, build and evaluate an adaptive conversational personalisation and voice interaction module that lets visually impaired and low-vision Sinhala users talk to the assistive reader in their own words and receive answers in the style each of them prefers.

Specific objectives.

1. To develop a Sinhala speech recognition interface that converts the user's voice commands into text.
2. To design and implement an intent detection mechanism that identifies the user's request from free-form Sinhala input.
3. To develop a user interaction analysis mechanism that records each user's interactions and preferences.
4. To implement personalised response generation that adjusts how answers are explained based on each user's preferences and history, by sending a personalised prompt to the answer-generation component.
5. To deliver the answer to the user as Sinhala speech through text-to-speech.
6. To evaluate the module with suitable metrics for recognition, intent detection and personalisation behaviour.

1.6 Research questions or hypotheses

- **RQ1.** Which freely available speech-to-text option gives usable Sinhala transcripts for short spoken reading commands from a normal laptop microphone?
- **RQ2.** Can a pipeline that translates Sinhala to English and then uses a small local language model detect user intent without any Sinhala intent dataset, and how does it compare with sending Sinhala directly to a small model and with a trained classifier?
- **RQ3.** Can a per-user online learning model predict a user's preferred answer style (Simple, Detailed or StepByStep) from that user's own interaction history, starting with no training data?
- **RQ4.** Which turns should such a model learn from, so that it does not collapse towards one style over time?

For RQ4 I tested one hypothesis:

- **H1.** If the model learns only from turns where the user gave explicit evidence of their preference (an explicit request, a supported flag, style keywords in their words, or a correction), and never from its own predictions, it will keep separate preferences for different users instead of drifting to the most frequent style.

1.7 Scope limitations and assumptions

In scope. Sinhala speech capture and transcription, translation to English, intent and flag extraction, routing of navigation commands, per-user answer-style learning and decision, correction handling, logging of interactions, the personalised prompt to Component 3, and connecting the answer to the system's speech output.

Out of scope. Reading the page (Components 1 and 2), retrieving content and generating the answer text (Component 3), and building a new Sinhala TTS voice. For speech output, my component uses the text-to-speech engine already implemented in the integrated system.

Limitations.

- Speech recognition uses Google's free web speech service, so it needs an internet connection.
- The test sets are small and the speech data comes from one speaker (myself), because no public labelled Sinhala voice-command or personalisation dataset exists. I therefore built my own test material and reported every output individually. Personalisation was evaluated with scripted users whose correct outcomes are known in advance, and testing it with real visually impaired users over a longer period remains future work.
- Three answer styles are supported: Simple, Detailed and StepByStep.

Assumptions.

- Each user identifies themselves with a unique user ID at the start of a session. No age, gender or other personal details are collected.
- A user's answer style is a stable but changeable property of that user, so recent signals should count more than old ones.
- When a user says clearly how they want the answer, that request is correct for that turn and the system must obey it.

1.8 Individual contribution

All of the following was my own individual work within the group project:

- Testing and comparing speech-to-text options on my own Sinhala recordings and integrating the selected one for live microphone input.
- Designing and building the intent detection stage (translation plus local language model), the open intent prompt, and the comparison against alternative approaches.
- Designing the personalisation method: per-user online models that combine the user's words with a recency-weighted summary of their confirmed history, together with the evidence-only learning rule and the four-tier decision order.

- Building the system-command filter, the correction mechanism, the flag-grounding check and the interaction logger.
- Moving the language model from a separate server application into the project as a local model file, so the module runs as one application.
- Building the demonstration and evaluation tools (live staged display, three-user comparison and intent debugging scripts) and running all experiments reported in this thesis.
- Defining the JSON contract between my component and Component 3.

1.9 Thesis structure

Chapter 2 reviews related work on Sinhala speech and language technology, intent detection, personalisation and online learning, and states the gap. Chapter 3 explains the research method, data, procedures and metrics. Chapter 4 presents the requirements, the architecture, the design alternatives I tested and the detailed design. Chapter 5 covers implementation, integration and testing. Chapter 6 reports the results. Chapter 7 discusses what the results mean, the contributions and the limitations. Chapter 8 looks at commercialisation and sustainability, and Chapter 9 concludes and lists future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

I reviewed work in four areas that my component depends on: (1) Sinhala speech recognition and language resources, (2) intent detection for low-resource languages, including translation-based and large language model (LLM) approaches, (3) user modelling and personalisation in conversational systems, and (4) online learning from streaming data, including the risk of a model learning from its own predictions. Sources were found through Google Scholar, the ACL Anthology, arXiv, IEEE Xplore and university repositories in Sri Lanka, using search terms such as "Sinhala speech recognition", "Sinhala intent", "low-resource intent detection", "user modelling", "online learning streaming", "cold start" and "confirmation bias pseudo-labelling". I also looked at the documentation of the tools I tested, because tool behaviour was a practical deciding factor in this research. The review is limited to work that helps explain or justify the design choices in this component; it is not a full survey of each field.

2.2 Domain and theoretical foundations

Speech recognition and its evaluation. Automatic speech recognition (ASR) converts audio into text. Modern multilingual systems such as Whisper are trained on very large amounts of weakly labelled audio and can transcribe many languages without fine-tuning [13]. Self-supervised speech encoders such as w2v-BERT 2.0, released with SeamlessM4T, are usually fine-tuned for a target language with a CTC head [14]. ASR quality is normally measured with word error rate (WER), the number of word substitutions, deletions and insertions divided by the number of reference words, and the character-level equivalent, character error rate (CER) [15]. For Sinhala, CER is useful because a word with one wrong vowel sign counts as a whole wrong word under WER.

Machine translation as a bridge. NLLB-200 is a single translation model that covers 200 languages, including Sinhala, and was built specifically to improve quality for low-resource languages [16]. Its distilled 600M-parameter version is

small enough to run on a laptop CPU. Translating a low-resource language into English before further processing lets later stages use English tools and models, which are far better resourced.

Intent detection and small language models. Intent detection maps an utterance to what the user wants to do. Classic systems train a classifier on a fixed set of labelled intents [8]. Large language models showed that a model can perform a task from instructions and a few examples without task-specific training [17]. Small instruction-tuned models such as Llama 3.2 1B [18] make this possible on ordinary hardware, especially when quantised and run with llama.cpp [19].

Text representation. TF-IDF weights each word by how often it appears in a text and how rare it is across texts [20]. It is simple, fast and easy to interpret, so the words that led to a style decision can be inspected and explained to the user.

User modelling and adaptive systems. Fischer describes user-adaptive systems as systems that change their behaviour based on observations of the user's characteristics or interactions [21]. A user model can be built from explicit feedback, where the user states a preference, or implicit feedback, which is inferred from behaviour. Explicit feedback is less frequent but more reliable, while implicit feedback is plentiful but noisy [22]. Any personalising system also faces the cold-start problem: what to do for a new user with no history [10].

Online learning. In online (incremental) learning, a model is updated one example at a time as data arrives, instead of being retrained on a fixed dataset. The River library provides online versions of common models, including feature extractors such as TF-IDF that grow their vocabulary as new words appear, and multiclass softmax (multinomial logistic) regression [23], [24]. Online learners have to deal with changes in the data over time, called concept drift [25]. In personalisation, a user changing their mind is a form of drift, which is why recent evidence should outweigh old evidence.

Learning from one's own predictions. When a model is trained on labels it produced itself, its errors become training data. In semi-supervised learning this is

called confirmation bias: pseudo-labelling can lock in early mistakes [11]. In recommender systems, a similar effect is described as a degenerate feedback loop, where the system's own outputs shape the data it later learns from and its view of the user narrows [12]. Both ideas are directly relevant to a personalisation model that learns live.

2.3 Related research

Sinhala speech recognition. Gamage et al. improved Sinhala ASR with an end-to-end LF-MMI model and reported a WER of 28.55% [5]. Work like this shows steady progress in recognition accuracy, but it focuses on transcription, not on what happens after the words are recognised. A survey of Sinhala NLP tools confirms that Sinhala resources are scattered and limited [2], and Ranathunga and de Silva show that Sinhala is one of the languages left behind in NLP research generally [3].

Sinhala intent detection. Karunanayake et al. identified intents in Sinhala and Tamil speech by first running an English phoneme-based recogniser and then classifying the phoneme output, which avoids needing a full Sinhala ASR [6]. Gupta et al. took a similar phone-based route and tested intent recognition on English, Dutch, Mandarin, Sinhala and Tamil with little training data [7]. Both approaches still need labelled examples for a fixed set of intents, and neither adapts to the user.

Large intent datasets. CLINC150 provides 150 intents and out-of-scope queries for English [8]. There is no comparable public Sinhala dataset for document-reading commands such as "summarise this" or "say it more simply", which is a key reason I avoided a design that depends on one.

Conversational systems and dialogue management. The RavenClaw framework separated task logic from dialogue control and showed how structured dialogue management improves spoken dialogue systems [26]. Commercial assistants such as Siri, Alexa and Google Assistant brought conversational voice interfaces to the public, but they are general-purpose and give largely the same responses to everyone [4].

Personalisation in dialogue. Persona-Chat showed that conditioning a dialogue model on a profile makes conversations more consistent and engaging [9]. That work personalises the system's own persona and conversation content, not the depth or structure of explanations for the listener. Recommender systems research has studied cold-start methods [10] and the difference between explicit and implicit feedback [22], which are the same two issues my personalisation stage has to handle.

Retrieval-augmented generation. RAG combines a retriever with a generator so answers are grounded in specific documents [27]. Component 3 of our system uses this idea. My component does not generate the answer itself; it shapes the request so the generator answers in the right style.

2.4 Existing systems products and techniques

Assistive reading tools and screen readers. Screen readers and camera-based reading apps convert text to speech and read it out. Their interaction is mostly one-directional: the user listens, and options such as speech rate are set manually in a settings menu. They do not learn that a particular user always asks for simpler explanations.

General voice assistants. Voice assistants can hold short conversations and answer questions [4], but they are not built around reading a specific document the user has just scanned, and they do not learn how a particular user wants things explained.

Techniques available for each stage. Table 2.1 summarises the techniques I considered for each stage of my component and what the literature or tool documentation says about them. These options became the alternatives tested in Chapters 4 and 6.

Table 2.1 Techniques considered for each stage of the component

Stage	Technique	Strength	Weakness for this project
Speech to text	Whisper (multilingual) [13]	No fine-tuning needed, many languages	Weak on short Sinhala commands in my tests
Speech to text	Fine-tuned Sinhala models (w2v-BERT 2.0, Whisper-small fine-tunes) [14]	Trained on Sinhala	Large downloads, small errors, cut off on long audio
Speech to text	Google web speech (SpeechRecognition library) [28]	Free to use, strong Sinhala output	Needs internet; audio leaves the device
Intent	Trained classifier on fixed intents [8], [6]	Fast, measurable	Needs labelled Sinhala data; fixed intent list
Intent	Small LLM directly on Sinhala	No training data	Small models understand Sinhala poorly
Intent	Translate to English, then small LLM [16], [18]	No Sinhala dataset needed; open intent labels	Two models, translation errors carry forward
Personalisation	Rules or keyword counters	Simple	Rigid, no learning
Personalisation	Batch retraining on stored logs	Familiar ML workflow	Needs a dataset and retraining cycles
Personalisation	Online learning per user [23]	Learns live, no upfront data	Must decide carefully what to learn from

2.5 Gap synthesis and conceptual framework

Putting the review together, existing work offers good pieces but not the combination this project needs. Sinhala ASR studies stop at transcription. Sinhala intent studies need labelled data and fixed intents. Personalisation research adapts personas or recommendations rather than explanation style, and online learning research warns that a model trained on its own outputs narrows over time. No reviewed system brings these together for Sinhala-speaking visually impaired users.

My conceptual framework follows from this. Each user turn is first converted into something a small model can understand (speech to Sinhala text, then English). The user's request is then read for two different things: **what** they want (intent) and **how** they want it (style signals). The "how" is decided in a fixed order of trust. What the user says explicitly always wins. Only when the user gives no signal does the per-

user model predict the style from that user's history. The model learns only from turns where the user gave real evidence, so the user, not the model, is the source of every training label. The result is a personalised prompt that Component 3 can follow. This framework is shown as an architecture in Chapter 4 (Figure 4.2).

3 RESEARCH METHODOLOGY

3.1 Research design

This research follows Design Science Research (DSR), as planned in my proposal. DSR builds an artefact to solve a real problem and then evaluates it, with the evaluation feeding back into the design [29]. The artefact here is the Component 4 module. I worked in build-and-evaluate cycles, one for each stage of the pipeline:

1. **Speech-to-text cycle.** Compare candidate STT options on recorded Sinhala commands and select one (RQ1).
2. **Intent detection cycle.** Build a translate-then-LLM approach, compare it with alternatives on the same test commands, and select one (RQ2).
3. **Personalisation cycle.** Design and build the per-user online learner, run it on real inputs, find its failure modes, change the design, and re-test (RQ3, RQ4).
4. **Integration cycle.** Restructure the pipeline into clear stages, move the language model into the project, and produce the output contract for Component 3.

The evaluation is mainly quantitative and controlled: the same inputs are fed to each alternative and the outputs are compared. For personalisation, where no labelled dataset exists, I used scripted scenarios in which the correct outcome is known in advance (Section 3.3.3). The early user feedback session (Section 3.2) gave qualitative input to the design.

3.2 Data participants or study materials

No external dataset was used for training the personalisation model. This is by design: the model starts empty for each user and learns only from that user's interactions. The materials used for selecting and evaluating the stages are listed in Table 3.1.

Table 3.1 Data and materials used in the research

Material	Size	Source	Used for
STT recording set A	5 Sinhala commands, 3.6–7.1 s each, 48 kHz, plus a 16 kHz re-recording	My own voice, laptop microphone	First STT comparison
STT recording set B	6 Sinhala commands of varied length, with hesitations and fillers, plus one joined file of all six (43 words)	My own voice	Final STT comparison (WER/CER)
Intent test commands	45 Sinhala commands over nine intents, with expected intents (Appendix A, Table A.1)	7 from the Google STT output of my recordings, plus 38 newly written commands checked by me and not used in any training	Comparing intent approaches
Generated command set	500 Sinhala reading commands over 17 intents, plus 30 added for the two weakest intents (530)	Prepared for this project	Training and testing the classifier alternative
Personalisation teaching script	3 users × 3 Sinhala requests, plus one neutral question (Appendix A, Table A.2)	Written for this project	Three-user personalisation test
Interaction logs	Every turn stored in TinyDB and CSV	Generated while the system ran	Analysing behaviour and failures

Participants. All recordings and test runs were done by me, so the speech data has one speaker. During the early prototype stage I also met a visually impaired person, explained how the prototype works and collected informal feedback on how they would like to talk to such a system. This was a single conversation, not a formal user study, and no personal data was recorded.

3.3 Experimental or study procedure

3.3.1 Speech-to-text comparison

1. Recorded Sinhala reading commands with a normal laptop microphone (sets A and B).

2. Ran each candidate model on the same files: OpenAI Whisper base, small and medium (language set to Sinhala, beam size 5, temperature 0); a fine-tuned Sinhala w2v-BERT model; a fine-tuned Sinhala Whisper-small model; and Google web speech through the SpeechRecognition library with language code si-LK [28].
3. Repeated the Whisper test after converting the audio to 16 kHz mono with stricter decoding settings, to check whether the audio format caused the failures.
4. Compared each transcript with the reference sentence. WER and CER were calculated for Whisper and Google speech recognition; for the two fine-tuned Sinhala models, transcripts were checked for exact match and inspected for errors.
5. Selected the option with the best transcripts, then built a live microphone version of it (ambient-noise calibration of 0.5 s, 8 s listening timeout, 15 s phrase limit).

3.3.2 Intent detection comparison

1. Built Approach 1: Sinhala text is translated to English with NLLB-200-distilled-600M, then Llama 3.2 1B returns a JSON object with an intent and personalisation flags.
2. Tried a closed intent list in the prompt first, then changed to an open prompt in which the model writes a short intent label in its own words, guided by examples.
3. Built Approach 2: the Sinhala text goes straight to Qwen2.5 1.5B with no translation.
4. Built Approach 3: a TF-IDF + LinearSVC classifier trained on the English translations of the generated command set.
5. Ran all approaches on the same 45 test commands and recorded the returned intent and the time taken. Matching was case-insensitive. None of the test commands were part of the classifier's training data.

3.3.3 Personalisation evaluation

There is no ground-truth dataset of "how this Sinhala user likes their answers", so I designed a scripted test where the right answer is known:

1. Reset all user models and clear the interaction log (`compare_users.py --reset`).
2. **Teaching phase.** Each of three users sends three Sinhala requests that clearly show one style: user_001 asks for simple and short answers, user_002 asks for more detail, and user_003 asks for step-by-step explanations (Appendix A, Table A.2).
3. **Test phase.** All three users then ask the same neutral question, "මෙහි ඇත්තේ කුමක්ද" ("what is in this?"), which contains no style words.
4. Record, for every turn, the decided style, which mechanism decided it (explicit intent, grounded flag, sentence keywords or the user's model), and whether the turn was learned from.
5. Repeat the test after each design change, and on both language model back ends (the Ollama server and the local model file).

In addition, I ran scenario tests in the live application: a system command (stop, go forward) to check that it is not learned from; a neutral question followed by "සරල කරන්න" ("simplify") to check the correction mechanism; and questions asking for depth, to check the keyword tier.

3.3.4 Diagnosis of the first personalisation version

The first version of the personalisation stage was run on real typed and spoken inputs, and its interaction log was analysed: how often each style was produced, how often the model was actually consulted, and which turns it was trained on. The findings led to the redesign described in Section 4.3.

3.4 Measures and evaluation metrics

Table 3.2 Evaluation metrics

Stage	Metric	Meaning
Speech to text	WER, CER [15]	Word and character errors against the reference sentence (lower is better)
Speech to text	Exact match	Whether the whole transcript matches the reference
Intent	Accuracy	Share of test commands whose returned intent matches the expected intent
Intent	Average latency (s)	Translation time plus model time per command
Intent (classifier)	Accuracy, macro-F1, 5-fold cross-validated macro-F1	Standard classification metrics on a held-out split of the generated set
Personalisation	Style match	Whether the style decided for a user matches the style that user taught
Personalisation	Decision source	Which tier decided the style; the test phase should be decided by "model"
Personalisation	Learned flag	Whether the turn was used as training evidence
Personalisation	Style distribution	Share of Simple, Detailed and StepByStep in the log, used to detect collapse
Routing	Command routing	Whether navigation commands bypass personalisation

3.5 Data analysis

Results are reported descriptively as counts, percentages and averages, which suit the purpose of each evaluation: comparing alternatives on the same inputs and selecting one for each stage. Every individual output is reported in the results tables and appendices, so each number can be traced back to the input that produced it. Speech-to-text output was compared with the reference sentences using word and character error rates. Intent detection was measured as accuracy and average latency, with all three approaches run on the same 45 commands. For the classifier alternative, I used a stratified 80/20 split with a fixed random seed (42) and 5-fold cross-validation, which is standard practice for that model type [30]. For personalisation, the analysis is per turn: each decision is traced to the tier that made it, which shows whether the user's model or a rule produced the style.

3.6 Ethics privacy security and responsible AI

- **Minimal personal data.** Users are identified only by a user ID. I decided not to collect age, occupation or any other personal information, and dropped an early idea of labelling users with personas (student, elderly, professional), because such labels cannot be justified from voice commands and are not needed.
- **Local storage.** Interaction logs and user models are stored locally on the machine running the module (TinyDB, CSV and a pickle file). Nothing is sent to a cloud database.
- **Third-party speech service.** Google speech recognition sends the recorded audio to Google's servers. Users should be told this, and it is listed as a limitation and a future-work item (an offline STT option).
- **User control.** A user's learned profile can be deleted at any time (`reset_user`), which supports the right to erasure under Sri Lanka's Personal Data Protection Act [31].
- **The user always wins.** When a user states how they want the answer, the system obeys it. The model is only used when the user gives no signal, and it never overrides an explicit request.
- **No self-reinforcement.** The model never learns from its own predictions, which prevents it from building a false picture of the user (Section 4.3).
- **Checking the language model.** Personalisation flags from the small LLM are only accepted when the user's own words support them (flag grounding), so a model error does not silently become a user preference.
- **Transparency.** Every decision returns its source (explicit intent, grounded flag, keywords or the model) in the pipeline output and the live display, so it is always possible to explain why a user received a certain style.
- **Ethics of testing.** All test recordings are my own voice. The feedback session with a visually impaired person was voluntary and informal, and no recordings or personal details were taken.

3.7 Reproducibility and research artefacts

The module is a Python project (`sinhala_assistive_reader/voice_interaction/`). The artefacts needed to reproduce the results are:

- **Code:** `stt/` (STT scripts), `intent_detection/` (Approaches 1–3, evaluation scripts and the LLM back end), `personalization/` (logger, diagnostic, style model, system commands, main flow), `app.py`, `compare_users.py`, `debug_intents.py` and `sync_logs.py`.
- **Data:** `data/test_samples.py` (45 test commands), `data/generate_samples.py` (generated commands), `data/db.json` and `data/interaction_logs.csv` (logs), and `intent_detection/intent_detection_results_all.json` (the intent comparison results).
- **Models:** `facebook/nllb-200-distilled-600M` from Hugging Face; `Llama-3.2-1B-Instruct-Q4_K_M.gguf` stored in `intent_detection/model/`; the classifier bundle `intent_classifier_bundle.joblib`.
- **Fixed settings:** LLM temperature 0 and 100 output tokens; classifier seed 42; personalisation settings in Table 4.6; library versions in `requirements.txt` (Table 5.1).
- **Commands:** `python compare_users.py --reset` reproduces the three-user test from an empty state; `python intent_detection/evaluate_all.py` reproduces the intent comparison; `python debug_intents.py` shows the translation, intent and flags for each teaching line.

Because Google speech recognition and LLM outputs can change slightly between runs or service versions, the exact outputs of every run reported here are kept in the logs and results files listed above.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements came from my proposal, the user requirements survey planned there, the feedback session with a visually impaired person, and the needs of the other components. Table 4.1 lists the functional requirements and Table 4.2 the non-functional requirements.

Table 4.1 Functional requirements

ID	Requirement
FR1	The module shall capture the user's spoken Sinhala request from a microphone and convert it to Sinhala text.
FR2	The module shall identify the intent of a free-form request without requiring the user to remember fixed phrases.
FR3	The module shall extract any explicit "how" signals in the request (detail level, language style, speed).
FR4	The module shall recognise navigation commands (go forward, stop) and route them to the reader without personalisation.
FR5	The module shall keep a separate preference profile for each user ID.
FR6	The module shall obey an explicit style request for the current turn.
FR7	When the user gives no style signal, the module shall predict the style from that user's own history.
FR8	The module shall treat a style request that contradicts the previous answer as a correction and relabel the previous turn.
FR9	The module shall log every interaction with its decided style, and report the source of each style decision.
FR10	The module shall produce a structured JSON prompt for Component 3 containing the intent, translation, style and prompt instruction.
FR11	The answer shall be delivered to the user as speech through the system's text-to-speech.

Table 4.2 Non-functional requirements

ID	Requirement
NFR1	**Low cost.** Use only free and open tools; no paid API per request.
NFR2	**Runs on a normal laptop CPU.** No GPU required.
NFR3	**One application.** The module should not depend on the user starting a separate server program.
NFR4	**No upfront dataset.** Personalisation must start working from the first interactions of a new user.
NFR5	**Privacy.** Store only a user ID and interaction logs, locally; no personal attributes.

ID	Requirement
NFR6	**Explainability.** Every style decision must report why it was made.
NFR7	**Robustness to model errors.** A wrong flag or label from the small language model must not become a stored preference.
NFR8	**Extensibility.** New navigation commands and new style words can be added without changing the learning code.

4.2 Component architecture within the complete solution

Figure 4.1 shows where Component 4 sits in the group system. Components 1 and 2 turn the captured page into structured article text, and Component 3 stores it and generates answers. Component 4 is the conversational layer between the user and Component 3: it receives the user's voice, sends Component 3 a personalised request, and returns the answer as speech.

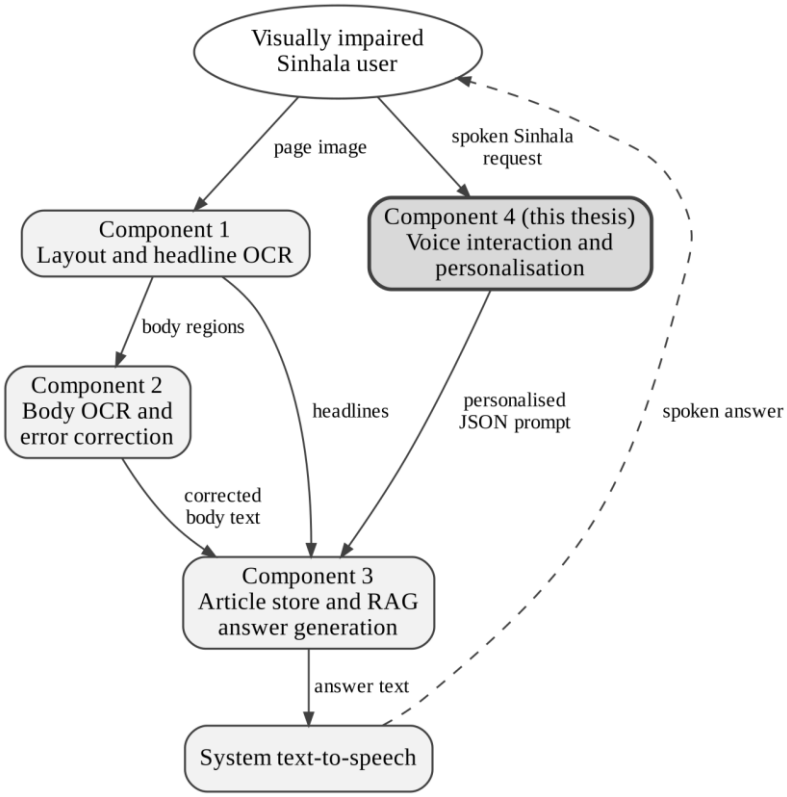


Figure 4.1 Component 4 within the group system

Inside the component, every user turn goes through the stages in Figure 4.2.

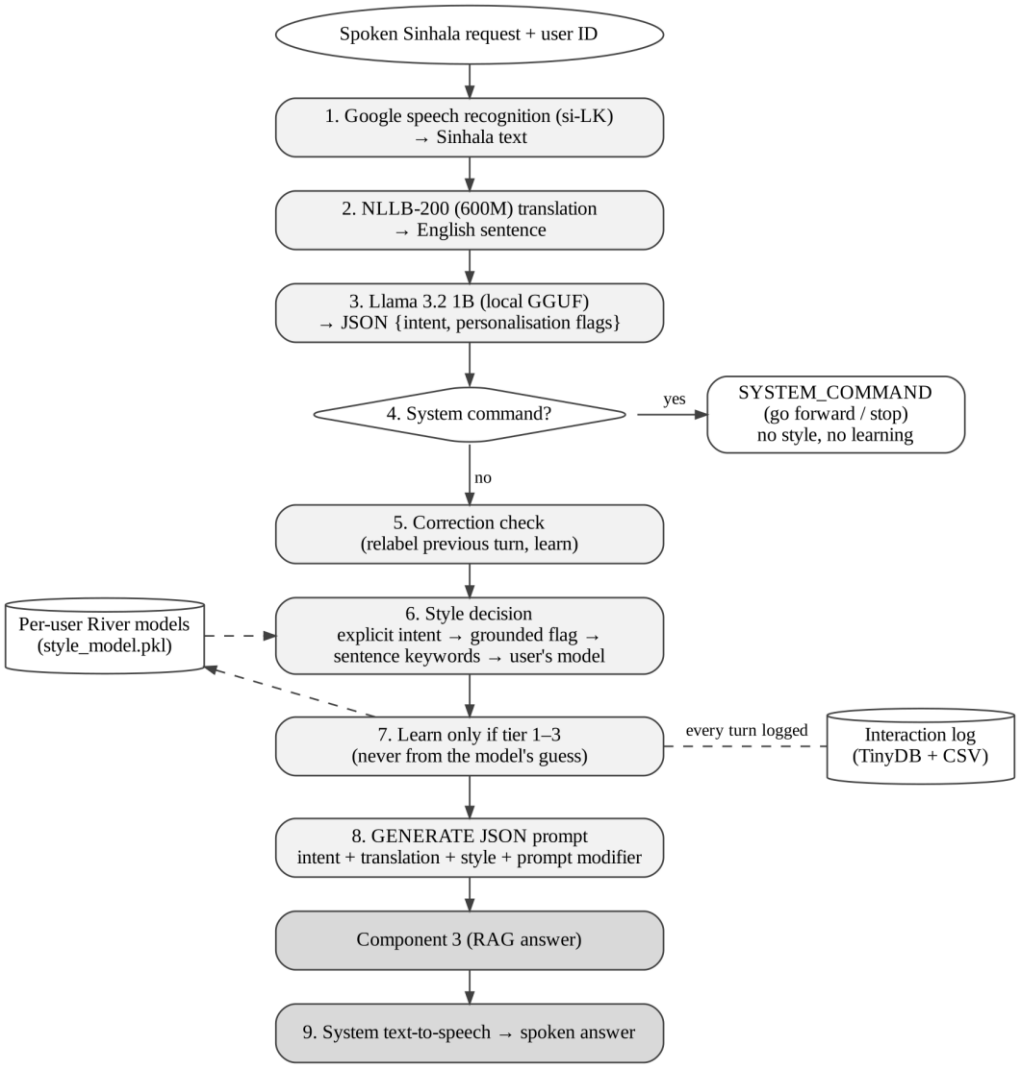


Figure 4.2 Per-turn flow of Component 4

1. **Speech to text.** Google speech recognition (si-LK) turns the spoken request into Sinhala text.
2. **Translation.** NLLB-200 translates the Sinhala text to English.
3. **Intent and flags.** Llama 3.2 1B reads the English sentence and returns JSON with an open intent label and personalisation flags.
4. **System-command check.** If the turn is a navigation command, it is routed straight to the reader with no personalisation and no learning.
5. **Correction check.** If this turn explicitly asks for a different style from the one just given, the previous turn is relabelled and learned from.

6. **Style decision.** The style is decided in a fixed order: explicit intent, then a grounded flag, then style keywords in the sentence, and only then the user's own model.
7. **Learning.** The user's model learns from this turn only if the style came from tiers 1–3.
8. **Prompt.** The style is converted into a prompt instruction and sent with the intent and translation to Component 3 as JSON.
9. **Speech output.** Component 3's answer is spoken to the user by the system's text-to-speech.

4.3 Design alternatives and decisions

Every main stage went through at least one alternative before the final design. Table 4.3 summarises the alternatives and the evidence behind each decision; the measured results are in Chapter 6.

Table 4.3 Design alternatives and decisions

Stage	Alternative	Decision	Reason
Speech to text	OpenAI Whisper base, small, medium	Rejected	Output was Romanised text, repetition loops or empty strings; WER of 100% or worse on every sample (Table 6.1)
Speech to text	Fine-tuned Sinhala w2v-BERT and Whisper-small models	Rejected	Close but with consistent small errors, and both cut off long audio
Speech to text	Google web speech via SpeechRecognition	**Selected**	0% WER on all six test sentences and 4.65% WER on the joined 43-word file; free to use
Intent	Closed list of intents in the prompt	Replaced	1 of 6 correct; a closed list cannot handle requests outside it
Intent	Approach 2: Sinhala directly into Qwen2.5 1.5B	Rejected	2.22% on the test commands; the small model does not understand Sinhala well enough
Intent	Approach 3: TF-IDF + LinearSVC classifier	Not used live	89% on held-out generated data but 62.22% on the test commands; fixed intent set

Stage	Alternative	Decision	Reason
Intent	Approach 1: NLLB + Llama 3.2 1B, open prompt	**Selected**	77.78% on the test commands; open labels; no Sinhala dataset needed
LLM hosting	Ollama server	Replaced	Needs a separate application to be started; kept only as a fallback
LLM hosting	Local GGUF file via llama-cpp-python	**Selected**	Runs inside the project as one application
Personalisation	Synthetic persona dataset + Random Forest	Rejected	Not real data; personas cannot be inferred from voice commands
Personalisation	Keyword counters with fixed update cycles	Rejected	Rule-based, no learning
Personalisation	LLM summarising the user's history	Rejected	No measurable model, slow, hard to evaluate
Personalisation	scikit-learn TF-IDF + SGDClassifier with partial_fit	Rejected	The TF-IDF vectoriser cannot be updated incrementally, so the feature size changes when it is refitted
Personalisation	River TF-IDF + one-vs-rest logistic regression	Replaced	Trains one binary model per class; softmax regression is natively multiclass
Personalisation	River TF-IDF + SoftmaxRegression, one global model, learning from every turn	Replaced	Collapsed towards Simple; no per-user profiles (Section 6.1.3)
Personalisation	**Per-user River model with history features, evidence-only learning**	**Selected**	Keeps separate preferences per user; the user is the only source of labels
Personalisation	Repeat-failure check on the retrieved chunk	Removed for now	Depends on chunk IDs from Component 3; future work

Three decisions shaped the final design more than the others.

Translate first, then use a small model. Small language models handle English far better than Sinhala. Translating first meant I could use an open intent prompt with a 1B-parameter model on a laptop, and avoid building a Sinhala intent dataset. The trained classifier was faster, but its accuracy dropped sharply when moving from its own generated sentences to new commands, and it can only output intents from its fixed list.

Learn from evidence only. The first personalisation version learned from every turn, including turns where the model had predicted the style itself. Analysing the interaction log of that version showed that about 85% of the answers were Simple, and that the model had been consulted on only a small share of the turns it was trained on. Most of its training labels came from rule-decided turns and from its own earlier predictions, so it was strongly biased towards the majority style and could not learn the less frequent styles, StepByStep in particular. This is the confirmation-bias problem described in the literature [11], [12]. Based on this analysis, I made the learning rule strict: the model learns only when the user has shown their preference, and the model's own prediction is never used as a training label.

Style belongs to the user, not to the sentence. A neutral question like "what is in this?" contains no style words, so a model that looks only at the words of the sentence has nothing to go on. I added a compact summary of the user's own confirmed history as features, so the model can answer the real question, "what does this user usually want?".

4.4 Detailed design

4.4.1 Intent and flag extraction

The intent prompt asks the model to return only a JSON object with two keys. The intent is a short English verb phrase that the model chooses freely, guided by examples (SUMMARIZE, EXPLAIN, SIMPLIFY, ELABORATE, STEP_BY_STEP, REPHRASE, IDENTIFY_CONTENT, READ_ALOUD, STOP, REPEAT, NEXT). The personalisation flags describe how the user wants the answer: speed (fast or slow), detail_level (brief, detailed or step_by_step) and language_style (simple or technical). The model runs with temperature 0 and a limit of 100 output tokens, and the reply parser removes code fences and falls back to the outermost braces if the model adds extra text.

Speed is a playback property, so it is reserved for the speech output stage and never used to decide the answer style. Detail level and language style are style signals, but

they come from a 1B model and are sometimes wrong, so they are checked against the user's words before they count (Section 4.4.3).

4.4.2 System commands

Navigation commands carry no information about answer style. "Stop" says nothing about whether a user prefers simple or detailed explanations. They are therefore checked before personalisation. A command is recognised if any of three signals match: the intent label (for example GO_FORWARD, GO_AHEAD, NEXT, CONTINUE for going forward, and STOP, PAUSE, HALT for stopping), the whole normalised utterance (for example "go ahead", "carry on", "stop it"), or a distinctive multi-word phrase inside it (for example "next page", "stop reading"). Single words such as "next" are only matched as the whole utterance, so a content request like "continue explaining this simply" is not mistaken for navigation. All punctuation is removed before matching, because the translation sometimes adds a detached full stop ("Go ahead ."). Commands are logged with no style, are never learned from, and return a SYSTEM_COMMAND route with an action description. New commands are added by adding one entry to the command registry.

4.4.3 The style decision

The style for a content request is decided in four tiers, from most to least trusted (Table 4.4).

Table 4.4 The four-tier style decision

Tier	Source	Rule	Learned from?
1	Explicit intent	The intent names a style. Simple: SIMPLIFY, SIMPLE, MAKE_SIMPLE(R), SUMMARIZE, SHORTEN, BRIEF. Detailed: ELABORATE, MORE_DETAIL(S), GIVE_DETAILS, FULL_DETAILS, DETAIL(ED), EXPAND, EXPLAIN_MORE. StepByStep: STEP_BY_STEP, STEPBYSTEP, STEPS, ONE_BY_ONE, WALK_THROUGH	Yes
2	Grounded flag	A detail_level or language_style flag counts only if it names the same style as the dominant style keywords in the English sentence	Yes
3	Sentence keywords	The sentence contains more keywords of one style than of the others (17 Simple, 18 Detailed and 9 StepByStep keywords); a tie gives no decision	Yes
4	The user's model	Used only when tiers 1–3 are silent: fewer than 3 confirmed signals gives cold start; top probability below 0.45 gives history fallback; otherwise the model's prediction	**No**

Intent labels are normalised before lookup (upper case, spaces and hyphens replaced with underscores), because the model invents its own variants such as "Simplify" or "MORE_DETAILS". Examples of the keywords are "simple", "easy", "brief", "short" and "summary" for Simple; "detail", "more", "full", "deep", "depth" and "complete" for Detailed; and "step", "one by one", "in order" and "walk me through" for StepByStep.

The grounding rule in tier 2 is deliberately strict. During testing, the model returned language_style: simple for "Give me the full details" and for "Can you explain this a little more?", two requests for more detail. Without grounding, these would have taught the user's model the opposite of what the user asked for. With grounding, a wrong flag is simply ignored, and the worst case is the same as having no flag. Flags are still extracted and passed to Component 3, so they keep their role in the prompt, but their influence on learning is minimised.

4.4.4 Correction

If the current turn has an explicit style intent (tier 1) and the previous turn of the same user received a different style, the previous turn is treated as wrong. Its log record is updated to the requested style with corrected = True, and the model learns

from it. For example, if a neutral question is answered in Detailed style and the user then says "සරල කරන්න" ("simplify"), the previous turn is relabelled Simple. A correction is strong evidence because the user has reacted to an actual answer.

4.4.5 The per-user model

Each user ID has its own River pipeline (Figure 4.3). Two groups of features are joined:

- **Text features:** TF-IDF over the English translation of the request [20].
- **History features:** the user's normalised history weights for each style (h_Simple, h_Detailed, h_StepByStep) and a confidence term n_conf = min(n, 20) / 20, where n is the number of confirmed signals.

The joined features go into a SoftmaxRegression classifier, which outputs a probability for each of the three styles [23], [24].

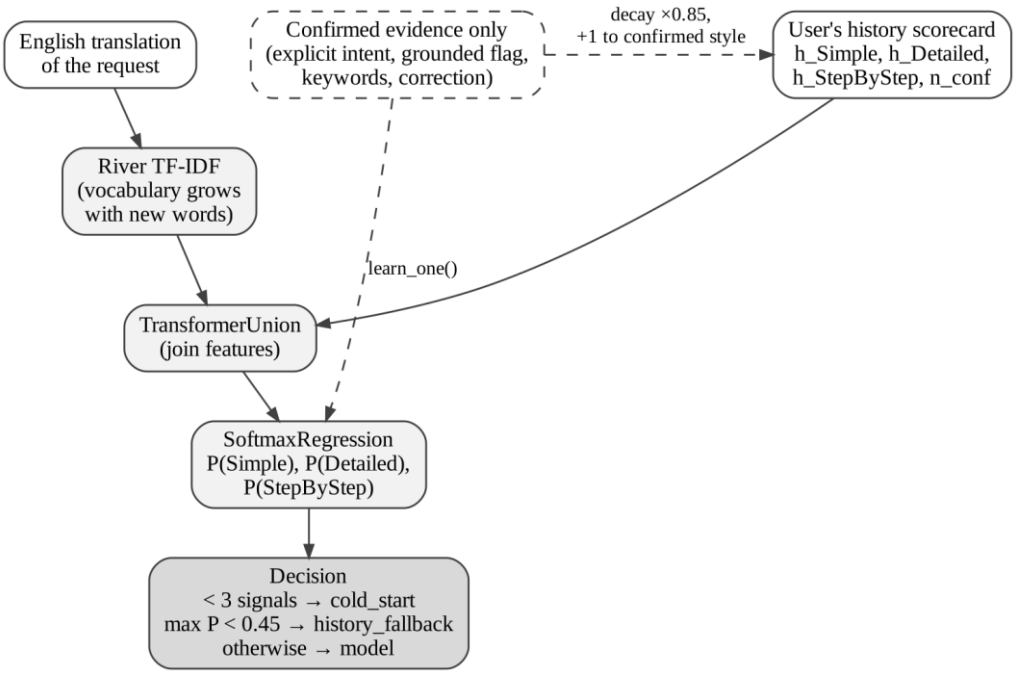


Figure 4.3 Structure of the per-user style model and its learning gate

The history weights work like a scorecard with fading memory. Each time the user gives a confirmed signal, all three weights are multiplied by a decay factor of 0.85 and the confirmed style gets +1. The weights are then normalised to sum to 1. Recent signals therefore count more than old ones, so a user who changes their preference is

followed instead of being out-voted by their own past. When the model learns, the features are built before the history is updated, so the model learns "given what I knew about this user, this was the right style".

Table 4.5 shows how the history weights change for a user who confirms StepByStep twice, then Simple once, then StepByStep again. The single Simple signal raises the Simple weight, but the user's repeated StepByStep preference stays dominant, and older signals count for less after each update.

Table 4.5 Worked example of the history scorecard update (decay 0.85)

Step	Simple	Detailed	StepByStep
Start (new user)	0	0	0
Confirm StepByStep	0	0	0 × 0.85 + 1 = 1.00
Confirm StepByStep	0	0	1.00 × 0.85 + 1 = 1.85
Confirm Simple	0 × 0.85 + 1 = 1.00	0	1.85 × 0.85 = 1.57
Confirm StepByStep	1.00 × 0.85 = 0.85	0	1.57 × 0.85 + 1 = 2.34
Normalised (model features)	0.27	0.00	0.73

When the user gives no signal, the model's decision has three possible sources, which are recorded with every turn:

- **cold_start** if the user has fewer than 3 confirmed signals: use the user's history majority, or the default style Detailed for a brand-new user;
- **history_fallback** if the model's top probability is below 0.45: use the history majority instead of a weak guess;
- **model** otherwise: use the model's prediction.

Detailed was chosen as the default because a user can always ask for a shorter answer, but a user who gets too little information may not know what is missing. All users' models are stored together in one pickle file and saved after every learning step, so learning survives restarts. Table 4.6 lists the settings. These were set as reasonable starting values and were not tuned, because tuning needs more real users than were available.

Table 4.6 Personalisation settings

Setting	Value	Purpose
Style classes	Simple, Detailed, StepByStep	The three answer styles
DEFAULT_STYLE	Detailed	Style for a brand-new user
MIN_CONFIRMED_FOR_MODEL	3	Signals needed before the model is trusted
CONFIDENCE_THRESHOLD	0.45	Below this, use history instead of the model
HISTORY_DECAY	0.85	How fast old signals fade
n_conf cap	20 signals	Point at which the confidence feature reaches 1

4.4.6 Prompt modifiers and the output contract

Each style is turned into a plain-English instruction for the answer-generation model (Table 4.7).

Table 4.7 Prompt modifiers sent to Component 3

Style	Prompt modifier
Simple	"Use very simple everyday words, short sentences, avoid technical terms."
Detailed	"Provide a thorough, in-depth explanation with full context, reasoning, and supporting detail. Do not shorten or oversimplify."
StepByStep	"Break the explanation into clear numbered steps, one action or idea per step."

For a content request the module returns a GENERATE payload. The example below is a real turn from the final three-user run. The flag from the language model says brief, but the style is Detailed because the grounding rule rejected that flag and the word "more" decided it:

```
{
  "route": "GENERATE",
  "intent": "EXPLAIN",
  "english_translation": "Can you explain this a little more?",
  "style_class": "Detailed",
  "prompt_modifier": "Provide a thorough, in-depth explanation with full context,
reasoning, and supporting detail. Do not shorten or oversimplify.",
  "personalization_flags": {"detail_level": "brief"},
  "correction_applied": null
}
```

For a navigation command it returns a SYSTEM_COMMAND payload with the fields route, command, action, intent and english_translation, for example command GO_FORWARD with the action "Advance to the next item".

4.4.7 Interaction log

Each turn is stored in the TinyDB table interaction_logs and mirrored to a CSV file that opens correctly in Excel (UTF-8 with BOM). Table 4.8 lists the fields. The field retrieved_chunk_id is kept as an empty placeholder for the article chunk that Component 3 will report back, which is needed for the future repeat-failure check.

Table 4.8 Interaction log record

Field	Content
user_id	The user's ID, for example user_001
timestamp	Time of the turn
sinhala_input	The Sinhala text from speech recognition
english_translation	The NLLB translation
intent	The intent returned by the language model
personalization_flags	The flags returned by the language model
retrieved_chunk_id	Placeholder for Component 3's chunk ID
style_class	The decided style (empty for system commands)
corrected	True if a later turn corrected this style

4.5 Standards and professional practice

- **Unicode Sinhala.** Sinhala text is handled as Unicode (block U+0D80–U+0DFF) throughout, and console output is forced to UTF-8 so Sinhala prints correctly on Windows.
- **JSON data interchange.** All stage outputs and the contract with Component 3 use JSON objects with fixed keys, so the other components do not depend on my internal code.
- **Python conventions.** Code follows PEP 8 naming and layout [32], with one responsibility per module (logging, evidence detection, model, commands, flow) and no printing inside the pipeline logic.

- **Software quality.** The design targets the ISO/IEC 25010 characteristics most relevant here: functional suitability, maintainability (modular files, a command registry, keyword lists that can be extended) and security/privacy (local data, minimal fields) [33].
- **Accessibility.** The design follows the spirit of accessibility guidelines such as WCAG [34]: the user can say what they want in their own words, can always override the system, and is never required to read anything on a screen.
- **Data protection.** Data collection is minimised and a user's profile can be deleted, in line with Sri Lanka's Personal Data Protection Act [31].
- **Version control of dependencies.** Library versions are listed in requirements.txt (pinned for the version-sensitive libraries), and the version-sensitive classifier bundle is tied to the scikit-learn version it was trained with.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The module was developed and tested on a standard personal laptop with no dedicated GPU, using VS Code, PowerShell and a Python 3.13 virtual environment. Google Colab (Tesla T4 GPU) was used only for the early STT comparisons and for training the classifier alternative, because those needed larger downloads or a GPU. Table 5.1 lists the main libraries.

Table 5.1 Technology stack

Purpose	Tool and version
Speech to text	SpeechRecognition 3.10.4 (Google web speech, si-LK), PyAudio 0.2.14
Translation	Hugging Face Transformers 5.7.0 [35], PyTorch 2.11.0, SentencePiece 0.2.1, facebook/nllb-200-distilled-600M [16]
Language model	Llama-3.2-1B-Instruct, Q4_K_M quantised GGUF (808 MB) [18], llama-cpp-python 0.3.35 (prebuilt CPU wheel) [19]; Ollama as fallback
Online learning	River 0.25.0 [23]
Storage	TinyDB 4.9.0 (JSON), CSV, pickle
Classifier alternative	scikit-learn 1.6.1 [30], joblib 1.4.2
Display	colorama 0.4.6

5.2 Component implementation

The code is organised by pipeline stage:

```
voice_interaction/
  stt/google_stt_live.py        microphone -> Sinhala text
  intent_detection/
    approach1_nllb_llm.py       NLLB + Llama intent and flags (selected)
    llm_backend.py              Llama from local GGUF file, or Ollama
    model/                      GGUF model file, classifier bundle
    approach2_direct_llm.py     comparison only
    approach3_trained_classifier.py, evaluate_all.py
  personalization/
    system_commands.py          navigation command registry and matcher
    diagnostic.py               evidence: intent, grounded flags, keywords
    style_model.py              per-user River models, history scorecard
    logger.py                   TinyDB + CSV logging
    main_flow.py                one full turn, returned stage by stage
  app.py                        live staged display
  compare_users.py              three-user personalisation test
  debug_intents.py              translation, intent, flags per line
  data/                         test samples, logs, saved models
```

Speech capture. `google_stt_live.py` opens the microphone, calibrates for background noise for 0.5 seconds, listens until the user stops speaking (8 s timeout, 15 s phrase limit) and calls Google speech recognition with the language code si-LK. Recognition errors and silence are reported back to the caller instead of crashing the pipeline.

Translation and intent. `approach1_nllb_llm.py` loads NLLB once at start-up, translates the Sinhala text with English set as the forced output language, and then sends the English sentence with the intent prompt to the language model. It returns the translation, intent, flags and the time spent in each step.

Local language model. At first the model ran on a separate Ollama server, which meant the user had to start another application before using the reader. I moved it into the project. `llm_backend.py` loads the GGUF file from `intent_detection/model/` with llama-cpp-python (context 2,048 tokens, temperature 0, 100 output tokens) and falls back to Ollama only if the library or file is missing. The back end can be forced with the LLM_BACKEND environment variable. Installing llama-cpp-python 0.3.2 from source failed on Windows with a C++ compiler error, so I installed the prebuilt CPU wheel (version 0.3.35) instead.

Personalisation flow. `main_flow.handle_voice_command(sinhala_text, user_id)` runs one full turn and returns four parts: stt_stage, intent_stage, personalization_stage and final_prompt. The core of the decision logic is short:

```
if intent_style:
    style, source, is_evidence = intent_style, "explicit_intent", True
elif flag_style:
    style, source, is_evidence = flag_style, "explicit_flag", True
elif keyword_style:
    style, source, is_evidence = keyword_style, "text_keywords", True
else:
    style, source = predict_style(user_id, english_text)
    is_evidence = False          # the model's own guess is never learned from

if is_evidence:
    learn_style(user_id, english_text, style)
```

Per-user model. `style_model.py` creates one River pipeline per user on first use. The features are built by joining TF-IDF on the text with the history weights, and the model is a SoftmaxRegression:

```
compose.Pipeline(
    compose.TransformerUnion(
        compose.Select("text") | feature_extraction.TFIDF(on="text"),
        compose.Select("h_Simple", "h_Detailed", "h_StepByStep", "n_conf"),
    ),
    linear_model.SoftmaxRegression(),
)
```

`learn_style` updates the model with `learn_one`, then applies the 0.85 decay to the history and adds 1 to the confirmed style, then saves all users to `data/style_model.pkl`. `predict_style` returns both a style and its source (cold_start, history_fallback or model). `reset_user` and `reset_all` clear profiles for demonstrations and for deleting a user's data.

Logging. `logger.py` writes every turn to TinyDB and re-exports the CSV, and it can update the style of the last turn or of a turn found by its timestamp (used by corrections). `sync_logs.py` moves logs between the JSON and CSV formats.

Demonstration tools. `app.py` shows each stage of a turn live, with a short delay between stages so a panel can follow it: Sinhala input, English translation, intent, flags, timings, the personalisation decision with its source, and the final JSON. It supports three input modes (live voice, typed Sinhala and the saved test samples) and three demo users or any custom user ID, and it shows each user's signal count and learned preference in the start menu. `compare_users.py` runs the three-user test of Section 3.3.3 and can save the comparison as an HTML table.

5.3 Integration with the other components

Input side. In the final build, the user speaks to the group's mobile app. The phone converts the speech to Sinhala text and sends the text to my service, which runs the rest of the pipeline from translation onwards. The live microphone mode with Google speech recognition (Section 5.2) is used in my standalone application and for testing.

Output side. For a content request, my module sends the GENERATE JSON (Section 4.4.6) to Component 3. Component 3 retrieves the relevant article content from its vector store and is designed to add the prompt_modifier to its generation prompt, so that the same question produces a simple, detailed or step-by-step answer depending on the user. For a navigation command, the SYSTEM_COMMAND payload is passed to the reader's navigation instead of Component 3.

Speech output. The answer text from Component 3 is spoken to the user through the text-to-speech engine already implemented in the integrated system, the same engine the reader uses for its other spoken output. I chose to reuse it rather than build a separate Sinhala TTS so that all audio in the product comes from one place. The speed flag is already extracted and kept for the speech stage, so adapting the speech rate per user can be added later without changing the rest of the pipeline.

Service boundary. In the group's integrated system, the components are connected as separate services. Component 4 has heavy libraries of its own (PyTorch, Transformers, llama.cpp, River), so it runs as an independent service that the reader calls over HTTP. This keeps library versions from clashing between components. If my service does not respond, the reader still handles basic commands (stop, next, previous, repeat) with a local fallback and marks those outputs as stubs, so unverified data is not passed on as if it came from the full pipeline.

Contract stability. The fields of the JSON contract have stayed fixed since the pipeline was restructured. The retrieved_chunk_id field is reserved in my log so Component 3 can return which chunk it used, which will allow the repeat-failure check to be brought back.

5.4 Verification and software testing

Testing was scenario-based, using the live application and the scripts, with the expected outcome of each test defined in advance. Table 5.2 lists the test cases and their final outcomes. Several of them failed when first run, and the failures led to the fixes listed in Table 6.7.

Table 5.2 Verification test cases

ID	Test	Expected	Outcome
T1	Say "නවත්වන්න" (stop)	SYSTEM_COMMAND STOP, no style, not learned	Pass
T2	Say "ඉදිරියට යන්න" (go forward)	SYSTEM_COMMAND GO_FORWARD	Failed first (model returned GO_AHEAD and translation had a detached full stop); pass after adding intent and phrase variants and removing all punctuation
T3	Explicit simplify request "සරල කරන්න"	Simple, source explicit_intent, learned	Pass
T4	Neutral question, then "සරල කරන්න"	Previous turn relabelled Simple with corrected = True	Pass
T5	New user asks a neutral question	Detailed by cold_start, not learned	Pass
T6	Request for depth ("Explain in depth")	Detailed from the evidence tiers	Failed first (StepByStep from the model, because "deep" and "depth" were missing from the keyword list); pass after adding them
T7	Three-user test (compare_users.py --reset)	Simple / Detailed / StepByStep for users 001 / 002 / 003, decided by the model	Failed first (user_002 taught Simple by wrong flags); pass after flag grounding and intent synonyms
T8	Restart the application	Learned profiles kept	Pass (pickle saved after every learning step)
T9	Local model back end check	Back end resolves to llama.cpp and returns parsable JSON	Pass ("Resolved: llamacpp", reply parsed)
T10	Three-user test on the local GGUF back end	Same result as on Ollama	Pass
T11	Sinhala output on the Windows console	No encoding errors	Failed first (UnicodeEncodeError); pass after forcing UTF-8 output

`debug_intents.py` was used as a regression check after every change to the evidence rules. It prints, for each of the nine teaching lines and the neutral question, the translation, intent, flags and the decided style, so any change in behaviour is visible immediately.

5.5 Deployment and operation

The module runs on a normal laptop or desktop without a GPU. Setting it up takes four steps:

1. Create a Python virtual environment and install the pinned requirements.
2. Install llama-cpp-python from the prebuilt CPU wheel index (`pip install llama-cpp-python --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cpu`).
3. Place `Llama-3.2-1B-Instruct-Q4_K_M.gguf` in `intent_detection/model/`. NLLB downloads automatically on first run (about 2.4 GB).
4. Run `python app.py` for the live demonstration, or start the module as a service for the integrated reader.

On start-up, NLLB and the language model are loaded once, so later turns do not pay the loading cost. User data stays in the `data/` folder of the machine running the module. An internet connection is needed only for speech recognition.

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Speech to text (RQ1)

Table 6.1 summarises the speech-to-text comparison. OpenAI Whisper did not produce usable Sinhala for these short commands. The base model transliterated the speech into Roman letters (for example "Meathavati ka kustra karanapulu vanda."), and the small and medium models fell into repetition loops such as "වවවව…" and "කරන්න් කරන්න්…". Converting the audio to 16 kHz mono and tightening the decoding settings did not help: the base model then returned empty text for four of five samples, and the larger models still repeated themselves.

The two fine-tuned Sinhala models were much closer. They recognised most words but made small, consistent mistakes. For example, for "කරුණාකර මෙය සාරාංශ ගත කර දෙන්න" the w2v-BERT model returned "කරුණා කරබ් සාරඥශ කත කර දැන්න" and the Whisper-small fine-tune returned "කරුණාකර මිය සාරාංශ ගත කර දෙන්න." Neither matched any of the five sentences exactly, and on the long joined file both stopped before the end.

Google speech recognition transcribed all seven files of set B. It made no errors on the six individual sentences and only two word errors on the 43-word joined file ("මේය" for "මෙය", and "ම්ම්ම්" for the filler "හ්ම්").

Table 6.1 Speech-to-text comparison on my Sinhala recordings

Model	Test set	Result
Whisper base	Set A (5 sentences)	WER 100% on all 5 (Romanised output)
Whisper small	Set A	WER 100% on all 5, CER 189–611% (repetition loops)
Whisper medium	Set A	WER 225% to 1,267% (word loops)
Whisper, 16 kHz audio and strict decoding	Set A	No improvement; base returned empty text for 4 of 5
Fine-tuned Sinhala w2v-BERT	Set A (16 kHz) and set B	0 of 5 exact matches; small character errors; long file cut off

Model	Test set	Result
Fine-tuned Sinhala Whisper-small	Set A (16 kHz) and set B	0 of 5 exact matches; small errors; stopped early on long file
Google speech recognition (si-LK)	**Set B, 6 sentences**	**WER 0.00%, CER 0.00% on each sentence**
Google speech recognition (si-LK)	**Joined file, 43 words**	**WER 4.65%, CER 1.64%; 7 of 7 files transcribed**

Answer to RQ1. Among the options I tested, Google speech recognition gave by far the most usable Sinhala transcripts for short spoken commands from a normal laptop microphone, including commands with hesitations and fillers. It was selected for the live pipeline.

6.1.2 Intent detection (RQ2)

The first version of Approach 1 used a closed list of intents in the prompt and got only 1 of 6 commands right (16.7%). The model forced requests into the nearest listed label, for example returning EXPLAIN for a rephrase request. After changing to an open prompt, where the model writes its own short intent label guided by examples, the results improved clearly.

Table 6.2 and Figure 6.1 compare the three approaches on the same 45 Sinhala test commands, covering nine intents. The per-command outputs of Approach 1 are in Appendix A (Table A.3).

Table 6.2 Intent detection approaches on the 45 Sinhala test commands

Approach	Correct	Accuracy	Average latency (s)
Approach 1: NLLB + Llama 3.2 1B (selected)	**35/45**	**77.78%**	**1.525**
Approach 2: Qwen2.5 1.5B, direct Sinhala	1/45	2.22%	5.195
Approach 3: NLLB + TF-IDF + LinearSVC	28/45	62.22%	0.925

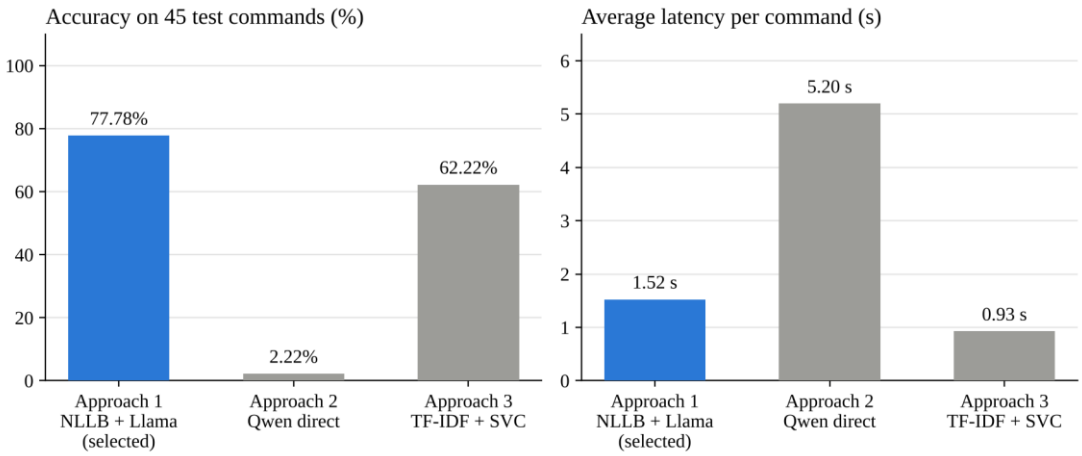


Figure 6.1 Accuracy and average latency of the intent approaches on the 45 test commands

The comparison ran on the laptop CPU with the local model file. The average latency of Approach 1 includes the first call, which took 8.6 s because the model was being loaded. Without that first call, Approach 1 took about 1.4 s per command: on average 0.92 s for translation and 0.61 s for the language model.

Approach 2 returned ANALYZE or ANALYZE_COMMAND for 30 of the 45 commands and got only one right. Without translation, the small model could not tell what the Sinhala requests meant. This comparison changes two things at once (no translation, and a different model), so it does not isolate the effect of translation alone, but it shows that a small model given raw Sinhala is not usable for this task, which supports the translate-first design.

The trained classifier (Approach 3) was the fastest. On a held-out 20% of the generated command set it scored well (Table 6.3), but on the test commands its accuracy fell to 62.22%, and 13 of its 17 errors had a confidence below 0.6. It failed on very short commands such as "කියවන්න" (read) and "දැන් කියවන්න එපා" (don't read now), and on long requests with an explanation before the actual request. The gap shows that sentences written for training do not cover the way people actually phrase requests, with fillers, hesitations and long explanations before the actual request.

Table 6.3 Classifier alternative on the generated command set (530 sentences, 17 intents)

Metric	Value
Held-out test accuracy (106 sentences)	89%
Held-out macro-F1	0.88
Mean macro-F1, 5-fold cross-validation	0.897
Vocabulary size (word unigrams and bigrams)	1,125
Average classification time	about 2 ms (excluding translation)
Accuracy on the 45 test commands	62.22%

Approach 1 missed 10 of the 45 commands, and the errors fall into three groups:

- **Lost meaning in translation (4 commands).** For example, sam_6, "ඒක හරියට තේරෙන්නේ නෑ වෙනත් විදිහකට කියන්න" ("I don't really understand it, say it another way"), became "It's just not clear.", and "ආයෙ" ("again") became "I'm going back.". Once the meaning is lost in translation, the language model cannot recover it.
- **Rephrase and simplify requests labelled EXPLAIN (4 commands).** Requests such as "say it in simple language" or "say it in a different way" were read as general requests to explain.
- **Navigation labels that were too specific (2 commands).** "Go to the other page" returned GO_TO_PAGE instead of NEXT, and one request for the next news item returned READ_ALOUD.

Most of these errors start in the translation step, not in the language model.

Answer to RQ2. Yes. Translating Sinhala to English and using a small local model with an open prompt detected the intent of 77.78% of the 45 test commands with no Sinhala intent dataset. It was clearly better than sending Sinhala directly to a small model (2.22%) and more accurate than the trained classifier (62.22%), at the cost of slightly higher latency.

6.1.3 Personalisation (RQ3 and RQ4)

The first version collapsed. The first personalisation version used one shared model and learned from every turn. Analysing its interaction log showed that about 85% of

the answers were Simple, with only a small share of other styles. The model was consulted on only a small share of the logged turns, but it was trained on all of them, including its own predictions. As a result, it was strongly biased towards the majority style and could not learn the less frequent styles, StepByStep in particular. In the first run of the three-user test (before flag grounding), user_002's requests for detail were labelled Simple through wrong flags from the language model, so user_002 ended up with a Simple profile. These findings led directly to the redesign described in Section 4.3.

The final version. Table 6.4 shows the three-user test across runs. After the evidence-only rule, per-user models, history features and flag grounding were in place, every teaching line was learned as intended, and the same neutral question received three different styles for the three users, each decided by that user's own model. The result was the same on the Ollama server and on the local model file.

Table 6.4 Three-user personalisation test across runs

Run	Back end	Teaching phase	Neutral question: user_001 / user_002 / user_003
First version	Ollama	user_002 learned Detailed, Simple, Simple	Simple / Simple / StepByStep
Final design	Ollama	All 9 teaching lines learned as intended	**Simple / Detailed / StepByStep**, all decided by the model
Final design	Local GGUF	All 9 teaching lines learned as intended	**Simple / Detailed / StepByStep**, all decided by the model

Table 6.5 shows the logged turns of the final run on the local model file. The decision sources for the teaching lines show that all three evidence tiers were used: explicit intents for user_001, flags confirmed by keywords and keywords alone for user_002, and explicit intents plus a confirmed flag for user_003.

Table 6.5 Final three-user run on the local model (from the interaction log)

User	Sinhala input	English translation	Intent	Decided style
user_001	සරල කරන්න	Simplify	Simplify	Simple
user_001	කරුණාකර මෙය ඉතාමත් සරලව පහදා දෙන්න	Please make this very simple.	Simplify	Simple

User	Sinhala input	English translation	Intent	Decided style
user_001	කෙටියෙන් කියන්න	Let me tell you briefly.	SUMMARIZE	Simple
user_002	තව විස්තර කරන්න	More details	EXPLAIN	Detailed
user_002	මෙය තව ටිකක් විස්තර කරන්න පුළුවන්ද	Can you explain this a little more?	EXPLAIN	Detailed
user_002	සම්පූර්ණ විස්තරය දෙන්න	Give me the full details	EXPLAIN	Detailed
user_003	පියවරෙන් පියවර පහදන්න	Step by step	Step by step	StepByStep
user_003	එකින් එක කියන්න	One by one .	STEP_BY_STEP	StepByStep
user_003	පියවර වශයෙන් විස්තර කරන්න	Step by step description	EXPLAIN	StepByStep
user_001	මෙහි ඇත්තේ කුමක්ද	What's wrong with that?	ASK_INQUIRY	**Simple**
user_002	මෙහි ඇත්තේ කුමක්ද	What's wrong with that?	ASK_INQUIRY	**Detailed**
user_003	මෙහි ඇත්තේ කුමක්ද	What's wrong with that?	ASK_INQUIRY	**StepByStep**

Two things in Table 6.5 are worth pointing out. First, in user_002's second line the language model returned the flag detail_level: brief for a request for more detail. The grounding rule rejected it because the sentence's own words ("more") point to Detailed, so the turn was still learned correctly. Second, the neutral question was translated as "What's wrong with that?". That translation is not accurate, but because it contains no style words, it still worked as a neutral test: the style could only come from each user's history.

After the scripted run, I asked a second neutral question, "මේකේ මොනවද තියෙන්නේ" ("What's in this?"), in the live application. user_002 received Detailed and user_001 received Simple, again matching what each had taught. When user_001 then asked "ගැඹුරින් පැහැදිලි කරන්න" ("Explain in depth"), the answer was Detailed, because the word "depth" in the request is explicit evidence. The explicit request overrode the user's learned preference for that turn, as the design requires.

Correction and commands. In a live session with a new user, "නවත්වන්න" (stop) was routed as SYSTEM_COMMAND STOP and not learned from. The neutral

question that followed got Detailed by cold start. The next turn, "සරල කරන්න" (simplify), was decided as Simple by explicit intent, and the previous turn was relabelled Simple with corrected = True, giving the user two confirmed Simple signals.

Style evidence on the test commands. Table 6.6 shows, as an analysis of the recorded outputs, how the four tiers would decide the style for the seven recorded test commands (sam_1 to sam_7), using the Approach 1 outputs for those commands. The language model attached a style flag to all 7 commands, but only 4 of these flags agreed with the meaning of the request. One was the opposite of the request (language_style: simple for "explain this a little more"), and two named a style for commands that contained no style request at all. Applying the decision rules to these outputs, the grounding rule would accept 2 flags, reject all 3 wrong ones, and also reject 2 correct flags that are not supported by keywords. Those two commands would still be decided correctly by the explicit-intent tier.

Table 6.6 Style evidence for the seven recorded test commands

ID	English translation	Flag from model	Flag accepted?	Decided style (tier)
sam_1	Summarize this	detail_level: brief	Yes	Simple (explicit intent)
sam_2	Can you explain this a little more?	language_style: simple	No (words say Detailed)	Detailed (keywords)
sam_3	Please explain this very simply and briefly…	language_style: simple	Yes	Simple (grounded flag)
sam_4	You can you tell me what I have here?	language_style: simple	No (no style words)	User's model
sam_5	I want to get this down without any details .	detail_level: brief	No (words say Detailed)	Simple (explicit intent)
sam_6	It's just not clear.	language_style: simple	No (no style words)	User's model
sam_7	Actually, I don't have time to listen… most important points…	detail_level: brief	No (no style words)	Simple (explicit intent)

sam_5 also shows a weakness of the keyword tier: "without any details" counts as a Detailed keyword because the keyword check does not understand negation. In this case the explicit-intent tier decided first, so the final style was still correct.

Answer to RQ3. Yes, within the scripted test. Starting with no data, each user's own online model learned that user's preferred style from three explicit signals and then predicted it correctly for a neutral question with no style words, for all three styles and on both model back ends.

Answer to RQ4 and H1. The results support H1. Learning from every turn, including the model's own predictions, led to a log in which about 85% of the answers were Simple, and in the first three-user run user_002 was pulled to Simple. With evidence-only learning, per-user models and grounded flags, the three users kept three different preferences. The model should learn only from turns where the user has shown their preference, and corrections, and never from its own guesses.

6.2 Product and technical results

Table 6.7 Defects found during testing and how they were fixed

Defect	Cause	Fix
Style almost always Simple	Model learned from its own predictions and from rule-decided turns; StepByStep had no examples	Evidence-only learning; STEP_BY_STEP added to the intent prompt
Same model for every user	Model had no user ID	One model per user ID
Requests for detail learned as Simple	Language model returned wrong flags	Flag grounding; intent synonyms
"Go forward" not recognised as a command	Model returned GO_AHEAD; translation had a detached full stop	Intent and phrase variants; all punctuation removed before matching
Requests for depth answered StepByStep	"deep" and "depth" missing from the Detailed keywords	Keywords added
Model reset on every restart	Model kept only in memory	Saved with pickle after every learning step
Detailed style gave no instruction	Empty prompt modifier	Clear Detailed instruction (Table 4.7)
Separate Ollama application required	Model served by an external server	Local GGUF file through llama-cpp-python
llama-cpp-python would not install	Source build failed with a C++ compiler error on Windows	Prebuilt CPU wheel
Sinhala text crashed the console	Windows console encoding	UTF-8 output forced in the logger

Other technical results:

- **One application.** With the model file inside the project, the module starts with one command and does not need Ollama to be running. The back-end check resolved to llama.cpp and returned parsable JSON.
- **Small footprint for personalisation.** The saved models for all users take 3.6 KB, and the database holding 15 interactions and 3 user entries takes about 6 KB. A new user adds only a small River pipeline and three history numbers.
- **Explainable decisions.** Every turn returns its decision source in the pipeline output and the live display, so it is always possible to show whether a style came from the user's words, a confirmed flag or the user's model.
- **Latency.** With the local model file, translation and intent detection took about 1.4 s per command after loading on the laptop CPU (Section 6.1.2). Speech recognition and the personalisation stage were not timed separately; personalisation only updates and queries a small per-user model.

6.3 User or stakeholder validation

Visually impaired user feedback. I met a visually impaired person, explained the prototype and how it would be used, and asked for their views. They liked being able to ask for things in their own words, for example asking the system to "explain this simply", instead of learning fixed commands. This supported the open intent design and the idea of letting the user's own requests drive personalisation. It was one informal conversation, not a formal usability study.

What is not yet validated. A formal study with visually impaired users, measuring whether personalised answers are actually easier to follow and how quickly the model learns over longer use, has not been carried out. It is the main future-work item (Section 9.4).

6.4 Integration results

- The module produces the GENERATE and SYSTEM_COMMAND payloads in the agreed JSON format for every turn, and the fields have been stable since the pipeline was restructured.
- Navigation commands are separated from content requests before personalisation, so the reader can act on them directly and they never reach Component 3.
- The answer is spoken through the system's existing text-to-speech, so no separate speech engine is needed for my component.
- In the integrated system the module runs as a separate service, and the reader keeps working for basic commands through a local fallback if my service is not available.

6.5 Summary of findings

- Google speech recognition gave the most usable Sinhala transcripts for short spoken commands (0% WER on six sentences, 4.65% on a 43-word joined file); Whisper was unusable on the same audio.
- Translating Sinhala to English and using Llama 3.2 1B with an open prompt detected intent for 77.78% of 45 test commands without any Sinhala intent data. Direct Sinhala input to a small model failed almost completely (2.22%), and a trained classifier dropped from 89% on generated data to 62.22% on the test commands.
- A personalisation model that learns from its own predictions collapses towards one style (about 85% of logged turns Simple).
- Learning only from user evidence, with one model per user and history features, kept three distinct preferences for three users, and a neutral question received Simple, Detailed and StepByStep answers decided by each user's own model.

7 DISCUSSION

7.1 Interpretation of findings

Why translate first. The intent results suggest that the language barrier matters more than the size of the model. A 1.5B model that received Sinhala directly could not identify a single intent, while a 1B model that received an English translation identified 77.78% of 45 commands. The two approaches also used different models and prompts, so this is not a perfectly controlled comparison, but the gap is large. Small models have seen far more English than Sinhala during training [3], so translation lets them use what they already know. The cost is that translation errors carry forward. Most of Approach 1's intent errors, and the inaccurate translation of the neutral question, came from the translation step. For the short commands this component mainly handles, the trade-off was clearly worth it.

Why a trained classifier was not enough. The classifier was fast and scored well on held-out generated data, but it lost about a quarter of that accuracy on the test commands, and it was weakest on very short commands. Real requests contain fillers, repeated words and long explanations before the actual request ("I don't have time to listen to the whole document, so…"). Sentences written to cover intents are cleaner than that. A fixed label set also cannot handle a new kind of request. The open prompt of Approach 1 handles both problems, which is why it was kept for the live pipeline.

Why the first personalisation version failed. The collapse towards Simple was not a bug in one line of code. It was a design problem: the model was being trained on labels it had produced itself. Once it leaned towards Simple, every Simple prediction became another Simple training example. This is the same pattern described as confirmation bias in pseudo-labelling [11] and as a degenerate feedback loop in recommender systems [12]. The fix also had to be a design rule, not a parameter change: only the user can produce a training label.

Why history features were needed. A neutral question has no style information in its words. A text-only model asked "what is in this?" can only fall back on whatever

class it has seen most often. Style preference belongs to the person, not the sentence, so the model needs to know about the person. Adding the user's own recency-weighted history as features turned the question from "what does this sentence want?" into "what does this user usually want?", which is the question personalisation is actually asking.

What the flags taught me. The flags from the 1B model sounded confident but were often wrong. Asking for "the full details" came back as language_style: simple. If those flags had been trusted, the system would have taught users the opposite of what they asked for. I did not want to remove the flags, because they are part of how the request is described to Component 3 and the speed flag is needed for speech output. Grounding them against the user's own words kept their value while making sure a wrong flag can never become a stored preference.

Rules and learning working together. The final design is a combination: rules decide when the user is clear, and the model decides only when the user says nothing about style. Each part covers the other's weakness. Rules alone cannot personalise a neutral question. A model alone can override what the user has just asked for, and it learns from its own mistakes. Together they respect the user's explicit request and still personalise the silent turns.

7.2 Comparison with literature baselines and existing solutions

Table 7.1 Comparison with related work

Work	Language	What it does	Adapts to the user?	Needs labelled training data?
Gamage et al. [5]	Sinhala	Speech recognition (WER 28.55%)	No	Yes (speech corpus)
Karunanayake et al. [6]	Sinhala, Tamil	Speech intent identification from phoneme ASR	No	Yes (intent labels)
Gupta et al. [7]	Including Sinhala, Tamil	Phone-based intent recognition and slot identification	No	Yes, small
Persona-Chat [9]	English	Dialogue conditioned on a persona	Persona of the speaker, not answer style	Yes
Commercial voice assistants [4]	Mainly English and major languages	General voice tasks	Limited, not explanation style	Yes (large internal data)
This component	**Sinhala**	**Speech to personalised prompt for a document reader**	**Yes, per-user answer style learned online**	**No Sinhala intent or personalisation dataset**

Compared with Sinhala speech and intent research, this component goes further than recognition and classification: it uses the recognised request to adapt the answer. Compared with Sinhala intent work [6], [7], it avoids the need for labelled intent data by translating and using an open prompt, at the cost of more computation per request. Compared with personalisation research [9], [10], it personalises a different thing, the style of explanation for the listener, and it handles the cold-start problem with a safe default and a minimum number of signals before trusting the model. Compared with work on online learning [23], [25], it applies an online learner per user and adds a strict rule on what counts as a training label, which the literature on confirmation bias suggests is necessary [11].

The numbers in this thesis cannot be directly compared with published accuracy figures, because the test sets are different and much smaller. The comparison is about what each approach is able to do and what data it needs.

7.3 Research contribution

1. **Evidence-only, per-user online learning of answer style.** A method in which each user has an online model that combines their words with a recency-weighted summary of their confirmed history, and learns only from explicit evidence (requests, grounded flags, style words and corrections), never from its own predictions.
2. **A four-tier decision order that keeps the user in control.** Explicit intent, then grounded flag, then sentence keywords, then the user's model, with every decision reporting its source.
3. **Flag grounding for small language models.** A simple check that accepts a style flag from a small model only when the user's own words support it, so model errors cannot become stored preferences.
4. **Evidence on translate-then-LLM intent detection for Sinhala.** A comparison on 45 Sinhala commands showing that translation plus a 1B model with an open prompt outperformed direct Sinhala input to a small model and a classifier trained on generated sentences.
5. **Documented failure analysis.** A concrete demonstration that a live personalisation model trained on its own predictions collapses towards one class, and how the design changes fixed it.

7.4 Product contribution

- A working Sinhala voice module that goes from speech to a personalised prompt and back to speech, running on a normal laptop CPU with free tools.
- A clear JSON contract that lets Component 3 apply the user's style without knowing anything about how it was learned.

- A navigation command filter that stops "stop" and "go forward" from being treated as questions or as preference signals, and that can be extended with one registry entry.
- A local language model setup that removes the need for a separate server application.
- Demonstration tools (live staged display, three-user comparison, intent debugging) that make each decision visible, which is useful for a panel, for testing and for future developers.

7.5 Limitations and threats to validity

- **Small, single-speaker test sets.** STT was tested on a handful of sentences in my voice, and intent on 45 commands. The results show clear differences between approaches, but the exact percentages would change with more speakers, accents, noise and devices.
- **Scripted personalisation evaluation.** The three-user test uses clear teaching sentences and a fixed neutral question. It shows that the mechanism works, but not how quickly real users' preferences are learned over weeks of use, how the model behaves when a user changes preference, or whether users notice the difference.
- **Translation errors.** Some short colloquial Sinhala requests were translated inaccurately (for example "What's wrong with that?" for "what is in this?"), and a lost meaning cannot be recovered later in the pipeline.
- **Keyword limits.** The keyword tier does not understand negation ("without any details" counts as a Detailed word), and its keyword lists are in English and fixed.
- **Untuned settings.** The default style, the minimum of three signals, the 0.45 confidence threshold and the 0.85 decay were set as reasonable starting values and not tuned on data.
- **An unexplained prediction.** In one live session the model predicted Simple for a user whose history weights favoured StepByStep (0.686) after a single Simple signal. With few signals, the text features and the history features can disagree, and this case needs further study.

- **Dependence on an online STT service.** Speech recognition needs the internet and sends audio to Google, which affects privacy and offline use.
- **Speech output not yet adaptive.** The system TTS is used as it is; speech rate and tone are not yet adjusted per user.
- **Features removed for now.** The repeat-failure check (telling apart a user who did not hear the answer from one who did not understand it) and implicit corrections ("that was too long") are not implemented.

7.6 Implications

For assistive technology in low-resource languages, these results suggest a practical route: use translation to reach strong English models, keep the models small and local, and spend the research effort on adapting to the person rather than on collecting large labelled datasets that do not exist. For anyone building a system that learns from users while it runs, the main lesson is to decide clearly what counts as evidence. Learning from your own guesses looks like learning, but it only reinforces what the model already believes. Finally, letting the user's explicit words always override the model is not only safer, it also produces the clean training signal the model needs.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

Primary users are visually impaired and elderly low-vision people who speak Sinhala. For them, the value of this component is that they can talk to the reader in their own words, and that the reader gets to know how they like things explained. A user who always asks for short, simple answers does not need to ask every time, and a user who wants detail gets detail by default.

A typical user, taken from my proposal, is Nimal Perera, a 42-year-old teacher with a visual impairment. He wants to ask questions about a document by voice and get an answer in Sinhala that matches his level, without relying on a screen.

Secondary users are organisations that serve these users: schools for visually impaired children, rehabilitation centres, libraries and disability organisations, which can run one installation for many users, each with their own profile.

The value in one sentence: a Sinhala reading assistant that listens, understands what you want, and learns how you like it explained, without paid services or a large dataset.

8.2 Market and competitive position

Screen readers and camera-based reading apps read text aloud, but they do not hold a conversation about the document or adapt their explanations to the listener. General voice assistants are conversational but are built for general tasks in major languages and treat every user the same [4]. The proposal originally imagined deployment on smart glasses connected to a laptop. As a group we moved to a smartphone app with a local or hosted back end, because specialised hardware is expensive for the people we want to reach, and almost everyone already has a phone. My component fits this model well: it runs on a normal computer without a GPU and stores only small files per user.

The competitive strengths of the component are Sinhala-first interaction, per-user personalisation that starts working without a dataset, local processing for everything

except speech recognition, and no per-request cost. Its weaknesses are the dependence on an online speech service and a delay of several seconds per request on a laptop CPU.

8.3 Business or sustainability model

The most realistic model for this kind of assistive technology in Sri Lanka is free access supported by institutions, rather than charging individual users who often cannot afford it.

- **Free core.** The app and the core system are offered free to individual users.
- **Institutional deployment.** Schools, rehabilitation centres and libraries run the back end on one of their own computers, serving many users on their local network. This removes cloud costs and keeps users' data on site.
- **Funding and partnerships.** Support from NGOs, disability organisations, government accessibility programmes or university research funds can cover maintenance and improvement.
- **Paid services for organisations.** Optional installation, training and support contracts for institutions can fund long-term maintenance.

The running cost of my component is low. Apart from the free speech recognition service, it uses no paid APIs, and the models run on hardware that organisations already own.

8.4 Scalability intellectual property and compliance

Scalability. Personalisation scales well: each user needs one small online model and three history numbers, and the saved models for all users in testing took only 3.6 KB. The heavier parts are the translation and language models, which are loaded once per server and shared by all users. For many simultaneous users, the main limit is CPU time for the language model, which can be handled by running more service instances or using a machine with a GPU.

Licences. Before any commercial use, the licences of the models need care:

- NLLB-200 is released for non-commercial use, so a commercial version would need a different translation model or a licence arrangement [16].
- Llama 3.2 is used under Meta's community licence, which allows commercial use subject to its terms [18].
- River, TinyDB, llama.cpp and the other libraries are open source under permissive licences.
- The free Google speech recognition endpoint used by the SpeechRecognition library is intended for testing and may be limited or withdrawn [28]; a production version would need Google's paid Cloud Speech-to-Text service or an offline Sinhala model.

Intellectual property. The original parts of this work are the personalisation design (evidence-only per-user learning, the four-tier decision order and flag grounding), the system-command design and the integration code. These could be released as open source to encourage further work on Sinhala assistive technology, which also fits the social purpose of the project.

Compliance. The module collects only a user ID and interaction logs, keeps them locally, and can delete a user's profile on request, in line with Sri Lanka's Personal Data Protection Act [31]. Users must be informed that their speech is processed by Google's speech service.

8.5 Contribution to the Sustainable Development Goals

- **SDG 10, Reduced Inequalities.** Target 10.2 aims to empower the social, economic and political inclusion of all, irrespective of disability [36]. A Sinhala voice reader that adapts to each person helps visually impaired users get information independently.
- **SDG 4, Quality Education.** Target 4.5 aims to ensure equal access to education for persons with disabilities [37]. Students with visual impairments can ask the reader to explain material simply or step by step, in the way that suits them.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research set out to let visually impaired and low-vision Sinhala users talk to an assistive reader in their own words and receive answers in the style that suits each of them. I built Component 4 as a pipeline of Google speech recognition, NLLB translation, a local Llama 3.2 1B model for intent and flags, a system-command filter, a per-user personalisation stage and a structured prompt to Component 3, with the answer spoken through the system's existing text-to-speech.

Each stage was chosen by testing alternatives. Google speech recognition gave usable Sinhala transcripts where Whisper did not. Translating to English before a small model detected intent for 77.78% of 45 test commands, while direct Sinhala input failed and a trained classifier did not generalise from generated to real phrasing.

The main finding is about personalisation. A model that learns from every turn, including its own predictions, slowly convinces itself that every user wants the same thing. A model that learns only from what the user has actually shown, keeps a separate profile per user and knows that user's history, can personalise even a neutral question. In the three-user test, the same question produced Simple, Detailed and StepByStep answers for three users, each decided by that user's own model, starting from no data at all. Throughout, an explicit request from the user is always obeyed.

9.2 Achievement of objectives

Table 9.1 Achievement of objectives

Objective	Status	Evidence
1. Sinhala speech recognition interface	Achieved	Google STT selected after comparison (Table 6.1); live microphone mode added to the application
2. Intent detection from free-form input	Achieved	NLLB + Llama 3.2 1B, 77.78% on 45 test commands (Table 6.2); runs locally
3. User interaction analysis and recording	Achieved	Per-turn logging with style and corrections; decision source reported per turn; per-user history scorecard
4. Personalised response generation	Achieved for the prompt; answer-level effect not yet measured	Per-user style decision and GENERATE prompt to Component 3 (Tables 6.4, 6.5)
5. Sinhala speech output	Partly achieved	Answers spoken through the system's existing TTS; per-user speech rate and tone not yet adapted
6. Evaluation	Partly achieved	STT, intent and scripted personalisation evaluation done; formal study with visually impaired users not yet done

9.3 Individual research and product contributions

Research contributions.

- An evidence-only, per-user online learning method for answer style that needs no upfront dataset and never trains on its own predictions.
- A four-tier style decision that keeps explicit user requests in control and reports the source of every decision.
- Flag grounding, which stops errors of a small language model from becoming stored user preferences.
- A comparison of intent detection approaches for spoken Sinhala commands, showing the advantage of translation plus a small model with an open prompt.
- An analysis of how and why a live personalisation model collapses when it learns from its own outputs.

Product contributions.

- A working Sinhala voice interaction module, from speech to personalised prompt to spoken answer, running on a laptop CPU with free tools and a local language model.
- A stable JSON contract with Component 3 and a separate route for navigation commands.
- Tools to demonstrate and test each decision of the pipeline.

9.4 Future directions

- **Study with visually impaired users.** Run a longer study with real users to measure how quickly preferences are learned, how the model follows a change of preference, and whether users find the personalised answers easier to follow.
- **Adaptive speech output.** Use the speed flag and each user's profile to adjust the speech rate and tone of the system TTS.
- **Repeat-failure detection.** Once Component 3 returns the chunk ID, bring back the check that tells a user who did not hear the answer (replay it more slowly) from one who did not understand it (explain it differently).
- **Implicit feedback.** Detect indirect corrections such as "that was too long" and use them carefully as weaker evidence [22].
- **Better translation for short commands.** Review the translation settings and test larger translation models or short-command post-editing, since translation caused the remaining intent error.
- **Tune the settings with real data.** Tune the default style, minimum signals, confidence threshold and decay once real usage data exists.
- **Offline speech recognition.** Fine-tune an open Sinhala ASR model on more speakers so the system can work without the internet.
- **Baseline comparison.** Compare the per-user online model with a language model that is simply given the user's history in its prompt, in terms of accuracy, consistency, latency and response to corrections.
- **Larger evaluation sets.** Build a multi-speaker Sinhala test set for STT and intent detection.

REFERENCES

[1] World Health Organization, "Blindness and vision impairment," Fact sheet, Feb. 2026. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment

[2] N. de Silva, "Survey on publicly available Sinhala natural language processing tools and research," arXiv:1906.02358, 2019.

[3] S. Ranathunga and N. de Silva, "Some languages are more equal than others: Probing deeper into the linguistic disparity in the NLP world," in Proc. 2nd Conf. Asia-Pacific Chapter of the ACL and 12th Int. Joint Conf. Natural Language Processing (AACL-IJCNLP), 2022, pp. 823-848.

[4] M. B. Hoy, "Alexa, Siri, Cortana, and more: An introduction to voice assistants," Medical Reference Services Quarterly, vol. 37, no. 1, pp. 81-88, 2018.

[5] B. Gamage, R. Pushpananda, T. Nadungodage, and R. Weerasinghe, "Improve Sinhala speech recognition through e2e LF-MMI model," in Proc. 18th Int. Conf. Natural Language Processing (ICON), 2021, pp. 213-219.

[6] Y. Karunanayake, U. Thayasivam, and S. Ranathunga, "Sinhala and Tamil speech intent identification from English phoneme based ASR," in Proc. Int. Conf. Asian Language Processing (IALP), 2019, pp. 234-239, doi: 10.1109/IALP48816.2019.9037702.

[7] A. Gupta, O. Deng, A. Kushwaha, S. Mittal, W. Zeng, S. K. Rallabandi, and A. W. Black, "Intent recognition and unsupervised slot identification for low-resourced spoken dialog systems," arXiv:2104.01287, 2021.

[8] S. Larson et al., "An evaluation dataset for intent classification and out-of-scope prediction," in Proc. EMNLP-IJCNLP, 2019, pp. 1311-1316.

[9] S. Zhang, E. Dinan, J. Urbanek, A. Szlam, D. Kiela, and J. Weston, "Personalizing dialogue agents: I have a dog, do you have pets too?," in Proc. 56th Annu. Meeting of the ACL, 2018, pp. 2204-2213.

[10] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, "Methods and metrics for cold-start recommendations," in Proc. 25th Annu. Int. ACM SIGIR Conf., 2002, pp. 253-260.

[11] E. Arazo, D. Ortego, P. Albert, N. E. O'Connor, and K. McGuinness, "Pseudo-labeling and confirmation bias in deep semi-supervised learning," in Proc. Int. Joint Conf. Neural Networks (IJCNN), 2020, pp. 1-8.

[12] R. Jiang, S. Chiappa, T. Lattimore, A. Gyorgy, and P. Kohli, "Degenerate feedback loops in recommender systems," in Proc. AAAI/ACM Conf. AI, Ethics, and Society (AIES), 2019, pp. 383-390.

[13] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. 40th Int. Conf. Machine Learning (ICML), 2023, pp. 28492-28518.

[14] Seamless Communication et al., "SeamlessM4T: Massively multilingual & multimodal machine translation," arXiv:2308.11596, 2023.

[15] A. C. Morris, V. Maier, and P. Green, "From WER and RIL to MER and WIL: Improved evaluation measures for connected speech recognition," in Proc. Interspeech, 2004, pp. 2765-2768.

[16] NLLB Team et al., "No language left behind: Scaling human-centered machine translation," arXiv:2207.04672, 2022.

[17] T. B. Brown et al., "Language models are few-shot learners," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877-1901.

[18] A. Grattafiori et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[19] G. Gerganov et al., "llama.cpp: LLM inference in C/C++," GitHub repository. [Online]. Available: https://github.com/ggml-org/llama.cpp

[20] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988.

[21] G. Fischer, "User modeling in human-computer interaction," User Modeling and User-Adapted Interaction, vol. 11, no. 1-2, pp. 65-86, 2001.

[22] G. Jawaheer, M. Szomszor, and P. Kostkova, "Comparison of implicit and explicit feedback from an online music recommendation service," in Proc. 1st Int. Workshop on Information Heterogeneity and Fusion in Recommender Systems (HetRec), 2010, pp. 47-51.

[23] J. Montiel et al., "River: Machine learning for streaming data in Python," Journal of Machine Learning Research, vol. 22, no. 110, pp. 1-8, 2021.

[24] C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.

[25] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACM Computing Surveys, vol. 46, no. 4, pp. 1-37, 2014.

[26] D. Bohus and A. I. Rudnicky, "The RavenClaw dialog management framework: Architecture and systems," Computer Speech & Language, vol. 23, no. 3, pp. 332-361, 2009.

[27] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459-9474.

[28] A. Zhang, "SpeechRecognition: Library for performing speech recognition, with support for several engines and APIs," Python Package Index, version 3.10.4. [Online]. Available: https://pypi.org/project/SpeechRecognition/

[29] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75-105, 2004.

[30] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[31] Parliament of the Democratic Socialist Republic of Sri Lanka, Personal Data Protection Act, No. 9 of 2022.

[32] G. van Rossum, B. Warsaw, and N. Coghlan, "PEP 8 - Style guide for Python code," Python Enhancement Proposals, 2001. [Online]. Available: https://peps.python.org/pep-0008/

[33] Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model, ISO/IEC 25010:2023, 2023.

[34] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/

[35] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. EMNLP: System Demonstrations, 2020, pp. 38-45.

[36] United Nations, "Goal 10: Reduce inequality within and among countries." [Online]. Available: https://www.un.org/sustainabledevelopment/inequality/

[37] United Nations, "Goal 4: Ensure inclusive and equitable quality education." [Online]. Available: https://www.un.org/sustainabledevelopment/education/

GLOSSARY

Answer style - How an answer is explained to the user: Simple, Detailed or StepByStep.

Cold start - The situation of a new user with too little history for the model to be trusted; the system uses a default or the user's history majority.

Correction - A turn in which the user explicitly asks for a different style from the one just given; the previous turn is relabelled and learned from.

Decision source - The tier that decided a turn's style: explicit_intent, explicit_flag, text_keywords, or the model (cold_start, history_fallback, model).

Evidence-only learning - The rule that the model learns only from turns where the user showed their preference, never from its own predictions.

Flag grounding - Accepting a style flag from the language model only when the user's own words support the same style.

GGUF - A file format for storing quantised language models so they can run with llama.cpp on ordinary hardware.

History scorecard - A per-user set of decaying weights for each style, updated with every confirmed signal and used as model features.

Intent - What the user wants to do, for example summarise, explain or go to the next item.

Online learning - Updating a model one example at a time as data arrives, instead of retraining on a fixed dataset.

Personalisation flags - Extra signals extracted from a request about how the user wants the answer: speed, detail level and language style.

Prompt modifier - The instruction added to Component 3's prompt to produce an answer in the chosen style.

System command - A navigation command such as stop or go forward, which is routed to the reader without personalisation.

APPENDICES

Appendix A Research instruments and extended results

Commands sam_1 to sam_7 are the Google speech recognition output of my recordings, and sam_8 to sam_45 were written for this test. None of them were used to train the classifier of Approach 3.

Table A.1 Intent test commands

ID	Sinhala command	Meaning	Expected intent
sam_1	මෙය සාරාංශ කරන්න	Summarise this	SUMMARIZE
sam_2	මෙය තව ටිකක් විස්තර කරන්න පුළුවන්ද	Can you explain this a bit more?	EXPLAIN
sam_3	කරුණාකර මෙය ඉතාමත් සරලව සහ කෙටියෙන් මට තේරෙන විදිහට පහදා දෙන්න	Please explain this very simply and briefly so I can understand	EXPLAIN
sam_4	ඔයා ඔයා මට මෙහි ඇත්තේ කුමක්දැයි කියන්න පුළුවන්ද	Can you tell me what is in this?	IDENTIFY_ CONTENT
sam_5	මට ඕනේ හ්ම් ඉතාමත් දිගු විස්තර නොමැතිව කෙටියෙන් වේගයෙන් මෙය පහදන්න	I want, hmm, this explained quickly and briefly without long details	SIMPLIFY
sam_6	ඒක හරියට තේරෙන්නේ නෑ වෙනත් විදිහකට කියන්න	I don't really understand it, say it another way	REPHRASE
sam_7	ඇත්තටම මට මේ සම්පූර්ණ ලියවිල්ලම අහගෙන ඉන්න වෙලාවක් නැහැ, ඒ නිසා මේකෙ තියෙන වැදගත්ම කරුණු ටික විතරක් තෝරලා අරගෙන මට පැහැදිලිව කියවන්න පුළුවන්ද	I don't have time to listen to the whole document, so can you pick only the most important points and read them clearly?	SUMMARIZE
sam_8	මේ පිටුවේ තියෙන දේ කෙටියෙන් සාරාංශයක් විදිහට දෙන්න	Give a short summary of what is on this page	SUMMARIZE
sam_9	අ මට ඕනෙ මේකෙ ප්‍රධාන අදහස් ටික විතරයි	Uh, I only want the main ideas of this	SUMMARIZE
sam_10	ඔක්කොම නෙවෙයි සාරාංශය විතරක් ඇති	Not all of it, just the summary is enough	SUMMARIZE
sam_11	මේ වචනෙ තේරුම මොකක්ද	What does this word mean?	EXPLAIN
sam_12	ඔයා ඔයා කිව්ව දේ මට තේරුනේ නෑ ඒක පහදලා දෙන්න	I didn't understand what you said, explain it	EXPLAIN
sam_13	මේ ආර්ථික වචන ටිකේ අදහස පැහැදිලි කරලා දෙන්න	Explain the meaning of these economic terms	EXPLAIN

ID	Sinhala command	Meaning	Expected intent
sam_14	මම මේක දෙපාරක් ඇහුවා ඒත් තාමත් ඔලුවට යන්නේ නෑ මේකෙ අදහස මට පැහැදිලි කරලා දෙන්න පුළුවන්ද	I listened twice but still don't get it; can you explain the idea?	EXPLAIN
sam_15	ලේසියෙන් කියන්න	Say it simply	SIMPLIFY
sam_16	හ්ම් මේ ලොකු ලොකු වචන නැතුව සාමාන්‍ය විදිහට කියන්න	Hmm, say it in a normal way without big words	SIMPLIFY
sam_17	මට මේ තාක්ෂණික වචන තේරෙන්නේ නෑ සරල භාෂාවෙන් කියන්න	I don't understand these technical words, say it in simple language	SIMPLIFY
sam_18	වෙනස් විදිහකට	In a different way	REPHRASE
sam_19	ඔය විදිහට නෙවෙයි වෙන ක්‍රමයකට ඒක කියලා දෙන්න	Not like that, explain it another way	REPHRASE
sam_20	මේ වාක්‍යය ටිකක් වෙනස් කරලා ආයෙ කියන්න	Change this sentence a little and say it again	REPHRASE
sam_21	මේ මොකක්ද	What is this?	IDENTIFY_ CONTENT
sam_22	හ්ම් මේ පිටුවෙ මොනවා ගැනද තියෙන්නෙ	Hmm, what is this page about?	IDENTIFY_ CONTENT
sam_23	මේ ලියවිල්ල මොන වගේ එකක්ද	What kind of document is this?	IDENTIFY_ CONTENT
sam_24	මගේ අතේ තියෙන මේ කොළේ මොනවද ලියලා තියෙන්නේ	What is written on this paper in my hand?	IDENTIFY_ CONTENT
sam_25	මම දැන් කැමරාව අල්ලගෙන ඉන්න මේ කොටසේ මොනවා ගැනද කතා කරන්නේ කියලා මට මුලින්ම කියන්න	I'm holding the camera now; first tell me what this section is about	IDENTIFY_ CONTENT
sam_26	ආයෙ	Again	REPEAT
sam_27	අ අ නැවත කියන්න	Uh, uh, say it again	REPEAT
sam_28	කලින් කියපු කොටස තව පාරක් කියන්න	Say the part you said before once more	REPEAT
sam_29	සද්දෙ නිසා මට ඇහුනෙ නෑ	I couldn't hear because of the noise	REPEAT
sam_30	සමාවෙන්න මගේ ෆෝන් එක රින්ග් වුණා ඒ නිසා ඔයා අන්තිමට කියපු ටික මට මඟ ඇරුණා ඒක ආයෙත් කියන්න	Sorry, my phone rang and I missed the last part you said; say it again	REPEAT
sam_31	ඇති	Enough	STOP
sam_32	හරි හරි නවත්තන්න	Okay, okay, stop	STOP
sam_33	දැන් කියවන්න එපා	Don't read now	STOP
sam_34	මට කෝල් එකක් එනවා ටිකකට නවත්තන්න	I'm getting a call, stop for a while	STOP

ID	Sinhala command	Meaning	Expected intent
sam_35	අද නම් මට ඇති වෙන්න ඇහුවා ඉතුරු ටික හෙට අහන්නම් දැනට කියවීම නවත්තන්න	I've heard enough for today, I'll listen to the rest tomorrow; stop reading for now	STOP
sam_36	ඊළඟ	Next	NEXT
sam_37	අ ඊළඟ පුවතට	Uh, to the next news item	NEXT
sam_38	අනිත් පිටුවට යන්න	Go to the other page	NEXT
sam_39	මේ පුවත ඇති ඊළඟ එක අහමු	This news item is enough, let's hear the next one	NEXT
sam_40	මේ පිටුවේ තියෙන ඔක්කොම මම අහලා ඉවරයි ඒ නිසා ඊළඟට තියෙන පිටුව පටන් ගන්න	I've heard everything on this page, so start the next page	NEXT
sam_41	කියවන්න	Read	READ_ALOUD
sam_42	හ්ම් මේ ලිපිය මුල ඉඳන් අගට කියවලා දෙන්න	Hmm, read this article from start to end	READ_ALOUD
sam_43	මේ පිටුවේ තියෙන දේ හඬ නඟලා කියවන්න	Read aloud what is on this page	READ_ALOUD
sam_44	මගේ බෙහෙත් කොළේ තියෙන දේ කියවලා දෙන්න	Read me what is on my prescription	READ_ALOUD
sam_45	මට ඇස් පේන්නෙ නැති නිසා මේ ලියුම මට කියවන්න බැහැ කරුණාකරලා ඒක මුලින්ම ඉඳන් කියවලා දෙන්න	I can't see, so I can't read this letter; please read it from the beginning	READ_ALOUD

Table A.2 Three-user teaching script

User	Teaching lines (Sinhala)	Style taught
user_001	සරල කරන්න; කරුණාකර මෙය ඉතාමත් සරලව පහදා දෙන්න; කෙටියෙන් කියන්න	Simple
user_002	තව විස්තර කරන්න; මෙය තව ටිකක් විස්තර කරන්න පුළුවන්ද; සම්පූර්ණ විස්තරය දෙන්න	Detailed
user_003	පියවරෙන් පියවර පහදන්න; එකින් එක කියන්න; පියවර වශයෙන් විස්තර කරන්න	StepByStep
All users	මෙහි ඇත්තේ කුමක්ද (neutral question, test phase)	None (decided by each user's model)

Table A.3 Approach 1 output for each test command (intent comparison run)

ID	NLLB English translation	Intent returned	Correct	Transl. (s)	LLM (s)
sam_1	Summarize this	SUMMARIZE	Yes	0.722	7.863 (model loading)

ID	NLLB English translation	Intent returned	Correct	Transl. (s)	LLM (s)
sam_2	Can you explain this a little more?	EXPLAIN	Yes	0.778	0.403
sam_3	Please explain this very simply and briefly as I understand it.	EXPLAIN	Yes	1.143	0.426
sam_4	You can you tell me what I have here?	IDENTIFY_ CONTENT	Yes	0.894	0.483
sam_5	I want to get this down without any details .	SIMPLIFY	Yes	0.948	0.460
sam_6	It's just not clear.	EXPLAIN	No	0.691	0.394
sam_7	Actually, I don't have time to listen to this whole document, so can I just pick out the most important points in it and read it clearly?	SUMMARIZE	Yes	2.553	0.598
sam_8	Give a brief summary of what is on this page	SUMMARIZE	Yes	1.041	0.449
sam_9	I just want the main ideas.	SUMMARIZE	Yes	0.701	0.433
sam_10	All is not enough.	SUMMARIZE	Yes	0.564	0.434
sam_11	What does this word mean?	EXPLAIN	Yes	0.608	0.395
sam_12	I don't understand what you said. Let it out.	EXPLAIN	Yes	1.134	0.441
sam_13	Explain the idea of these economic words	EXPLAIN	Yes	0.838	0.428
sam_14	I've asked this twice but it still doesn't come to mind. Can you explain the idea to me?	EXPLAIN	Yes	1.936	0.512
sam_15	Easy to say .	SIMPLIFY	Yes	0.495	0.420
sam_16	Hmm these are big big words without saying it normally	EXPLAIN	No	0.959	0.437
sam_17	I don't understand these technical words to say in simple language.	EXPLAIN	No	1.207	0.432
sam_18	In a different way	REPHRASE	Yes	0.462	0.488
sam_19	Don't tell me it's not like that.	EXPLAIN	No	1.033	0.454
sam_20	Change the sentence a little and say it again .	UNKNOWN	No	0.905	0.580

ID	NLLB English translation	Intent returned	Correct	Transl. (s)	LLM (s)
sam_21	What is this?	IDENTIFY_ CONTENT	Yes	0.493	0.408
sam_22	What's on this page about?	IDENTIFY_ CONTENT	Yes	0.761	0.444
sam_23	What kind of document is this ?	IDENTIFY_ CONTENT	Yes	0.683	0.446
sam_24	What is written in this paper in my hand	IDENTIFY_ CONTENT	Yes	0.817	0.451
sam_25	I'm gonna hold the camera now and tell me what you're talking about in this section.	IDENTIFY_ CONTENT	Yes	1.713	0.544
sam_26	I'm going back.	PREVIOUS	No	0.635	0.367
sam_27	I'll tell you again.	REPEAT	Yes	0.707	0.392
sam_28	Previously on " Tell Me Another Way "	PREVIOUS	No	0.844	0.423
sam_29	I didn't hear it because of the noise.	REPEAT	Yes	0.988	0.438
sam_30	Sorry my phone rang so I missed the last thing you said.	REPEAT	Yes	1.149	0.467
sam_31	That's enough.	STOP	Yes	0.532	0.345
sam_32	Okay, okay, stop it.	STOP	Yes	0.688	0.347
sam_33	Don't read it now.	STOP	Yes	0.691	0.351
sam_34	I got a call coming in. Stop for a while.	STOP	Yes	1.030	0.419
sam_35	I've had enough today. I've heard the rest tomorrow. I'll ask you.	STOP	Yes	1.730	0.414
sam_36	Next .	NEXT	Yes	0.337	0.329
sam_37	A to the next news	NEXT	Yes	0.557	0.365
sam_38	Go to the other page	GO_TO_PAGE	No	0.551	0.430
sam_39	This is the news. Let's hear the next one.	READ_ALOUD	No	1.115	0.504
sam_40	I've heard everything on this page so start the next page.	NEXT	Yes	1.192	0.410
sam_41	Read it	READ_ALOUD	Yes	0.337	0.449
sam_42	Let me read this article from the beginning .	READ_ALOUD	Yes	0.826	0.499

ID	NLLB English translation	Intent returned	Correct	Transl. (s)	LLM (s)
sam_43	Read aloud what's on this page.	READ_ALOUD	Yes	0.910	0.473
sam_44	Read me what's in my medicine book.	IDENTIFY_ CONTENT	No	0.926	0.451
sam_45	I can't read this letter because I can't see. Please read it first.	READ_ALOUD	Yes	1.570	0.544

Appendix B Academic integrity and AI use disclosure

Table B.1 Use of generative AI and automated tools

Tool	Purpose	How the output was checked	Parts affected
General-purpose AI chat assistants	Explaining concepts (for example TF-IDF and online learning), discussing design options, helping debug errors, and drafting parts of the code that I then integrated and modified	I made the design decisions after discussing the options; some starting values (for example the personalisation thresholds) and 30 additional classifier training sentences came out of these discussions and were adopted by me; all code was run and tested on my own machine, and the results in this thesis come from my own runs, logs and results files	Chapters 4 and 5 (design and implementation), project code
Google speech recognition, NLLB-200, Llama 3.2 1B, Qwen2.5 1.5B	Components of the system under study	Evaluated as described in Chapter 6	The artefact itself

I remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.